

UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA Informatică Română

LUCRARE DE LICENȚĂ

**Detectarea automată a numerelor de
înmatriculare folosind tehnici clasice
de viziune artificială și clasificatori
statistici**

Conducător științific
Conferențiar, Dr. Bufnea Darius

*Absolvent
Pușcaș Raul-Andrei*

ABSTRACT

Automatic license plate recognition (ALPR) has become a core building block of intelligent transportation systems, underpinning applications as diverse as traffic law enforcement, electronic toll collection, stolen-vehicle detection and the management of restricted-access areas such as parking facilities. At its essence, an ALPR system extracts a vehicle’s unique alphanumeric identifier from an image and converts it into machine-readable text — a task that remains challenging under unconstrained conditions, owing to variations in illumination, plate format, font, viewing angle and partial occlusions.

This thesis proposes such a system, named IORA, designed around a modular architecture and relying solely on classical computer vision techniques for the image-processing stages and on a statistical Support Vector Machine (SVM) classifier for character recognition. The work targets a concrete deployment scenario, an automated access-control point for a parking facility in Romania, which imposes two requirements: the robust handling of the many plate formats in circulation, from standard and temporary plates to diplomatic and institutional ones, and an execution time low enough to actuate the barrier within one to two seconds of a vehicle’s arrival.

Following the canonical ALPR pipeline, the system is organised into three sequential stages. Plate localization combines the Sobel operator with morphological operations and geometric filtering to isolate candidate regions; character segmentation separates the individual symbols through contour analysis guided by a priori knowledge of plate geometry; and recognition assigns a label to each character using a multi-class SVM trained on HOG descriptors. By avoiding deep neural networks altogether, the pipeline is able to run on modest hardware without GPU acceleration.

The system is evaluated both end-to-end and per stage, on a heterogeneous dataset spanning Romanian and international plates. The results show that, when classical methods are guided by domain-specific a priori knowledge, they remain a competitive and resource-efficient alternative for real-time license plate recognition, while also exposing the limitations that motivate further work.

Cuprins

1	Introducere	1
2	Metode de localizare a obiectelor	3
2.0.1	Metode bazate pe caracteristici cromatice	3
2.0.2	Metode bazate pe conturul obiectelor	5
2.0.3	Metode bazate pe textura obiectelor	9
3	Segmentarea imaginii	12
3.0.1	Conectivitatea pixelilor	12
3.0.2	Analiza proiecțiilor	12
3.0.3	Utilizarea informațiilor apriori privind aspectul caracterelor .	14
3.0.4	Analiza conturului caracterelor	15
4	Recunoașterea caracterelor	17
4.1	Potrivire de șabloane	17
4.2	Clasificatori statistici	18
4.2.1	Mașini pe suport vectorial	19
4.2.2	Rețele neuronale artificiale	24
5	Implementarea sistemului IORA	28
5.1	Arhitectura sistemului	28
5.1.1	Diagrama de componente	29
5.1.2	Actori și cazuri de utilizare	29
5.1.3	Diagrama de clase	30
5.1.4	Decizii de proiectare	31
5.2	Tehnologii	32
5.2.1	Nucleu	32
5.2.2	Nivelul de aplicație	32
5.3	Detalii tehnice	32
5.3.1	Modulul de intrare	33
5.3.2	Fluxul de procesare	34

5.3.3	Nivelul de aplicație	40
6	Metrici și experimente	41
6.1	Seturi de date	41
6.1.1	Setul de evaluare	41
6.1.2	Setul de antrenare a clasificatorului SVM	43
6.2	Testare și metrici	46
6.2.1	Performanța componentei de recunoaștere a caracterelor . . .	46
6.2.2	Performanța întregului sistem	48
6.3	Experimente	51
6.3.1	Comparația abordărilor de localizare	51
6.3.2	Comparația abordărilor de segmentare	52
6.3.3	Îmbunătățiri progresive ale fluxului de recunoaștere	52
7	Concluzii și perspective de viitor	56
7.1	Concluzii	56
7.2	Perspective de viitor	57
	Bibliografie	58

Capitolul 1

Introducere

Creșterea constantă a numărului de vehicule aflate în circulație a făcut ca identificarea automată a acestora să devină o componentă esențială în infrastructura modernă de transport. Lucrarea de față propune un sistem de recunoaștere automată a numerelor de înmatriculare (ALPR — *Automatic License Plate Recognition*) construit pe o arhitectură modulară, bazată exclusiv pe tehnici clasice de viziune artificială pentru etapele de procesare a imaginii și pe un clasificator statistic de tip SVM (*Support Vector Machine*) pentru recunoașterea caracterelor.

Studiul a fost motivat de cerințele concrete ale unui sistem de control al accesului auto într-o parcare situată pe teritoriul României. Acest context impune două constrângeri clare. În primul rând, sistemul trebuie să recunoască fiabil o varietate largă de formate: plăcuțe uzuale –două litere –două cifre –trei litere, plăcuțe provizorii cu serie roșie, plăcuțe pentru autoturisme noi în format extins, plăcuțe diplomatice (CD, TC, CO) și plăcuțe ale Ministerului Afacerilor Interne. În al doilea rând, întregul flux de procesare, de la achiziția cadrului până la validarea numărului trebuie să aibă un timp de execuție redus, de ordinul sutelor de milisecunde, astfel încât bariera de acces să fie acționată în cel mult 1–2 secunde de la momentul sosirii vehiculului.

Soluția propusă structurează procesul în trei etape secvențiale. Etapa de localizare izolează regiunea plăcuței într-o imagine de intrare folosind operatorul Sobel pentru extragerea contururilor verticale, urmat de operații morfologice și filtre geometrice care elimină regiunile candidate necorespunzătoare. Etapa de segmentare separă caracterele individuale prin analiza contururilor și a informațiilor apriori. În final, etapa de recunoaștere clasifică fiecare caracter segmentat printr-un model SVM multi-clasă antrenat pe un set de caractere etichetate. Întreg ansamblul este implementat fără a recurge la rețele neuronale profunde, ceea ce permite execuția pe hardware modest, fără accelerare GPU.

În comparație existente, soluția prezentată se evidențiază prin două contribuții. Prima este reziliența la variațiile de mediu și de format precum diferențe de lumi-

nozitate, fonturi neomogene între seriile de plăcuțe, dimensiuni variabile și ocluzii parțiale, caracteristică obținută prin alegerea unor tehnici robuste în fiecare etapă a fluxului. A doua contribuție este nivelul de optimizare adus fiecărui modul, astfel încât sistemul să respecte constrângerea de timp real pe o platformă *embedded*.

Lucrarea este organizată în șase capitole. Primele trei capitole au caracter teoretic și prezintă fundamentele necesare. Capitolul 2 tratează problema localizării obiectelor de interes într-o imagine și introduce, ca aplicație particulară, metodele folosite în literatură pentru localizarea plăcuței de înmatriculare. Capitolul 3 prezintă tehnicile de segmentare a regiunilor identificate, cu accent pe separarea caracterelor. Capitolul 4 discută clasificarea caracterelor, pornind de la metodele bazate pe șabloane și culminând cu fundamentele teoretice ale clasificatorului SVM. Capitolul 5 constituie contribuția practică a lucrării: descrie arhitectura, deciziile de implementare și detaliile tehnice ale modulelor menționate în paragraful precedent. Capitolul 6 prezintă evaluarea experimentală a sistemului IORA, inclusiv metricile obținute pe setul de testare și rezultatele obținute în condiții reale de operare. Capitolul 7 sintetizează concluziile, evidențiază limitările soluției actuale și schițează direcțiile de dezvoltare ulterioară.

Capitolul 2

Metode de localizare a obiectelor

Detectarea unui obiect de interes într-o imagine este una dintre cele mai abordate teme din viziunea artificială. Pentru a atinge acest scop variantele sunt numeroase, toate având o caracteristică comună: dependența de trăsăturile obiectului vizat. Un exemplu practic îl reprezintă detecția fețelor în imagini: pentru ca o regiune să fie identificată drept față, aceasta trebuie să prezinte o formă aproximativ ovală, în interiorul căreia să se regăsească două zone simetrice corespunzătoare ochilor, dispuse în treimea superioară, o structură verticală alungită asociată nasului, plasată central, și o regiune orizontală sub aceasta, corespunzătoare gurii. Fiecare obiect poate fi astfel caracterizat printr-un set de trăsături distinctive geometrice, cromatice sau texturale care permit diferențierea sa de restul scenei.

În contextul lucrării de față, obiectul de interes îl reprezintă plăcuța de înmatriculare, a cărei localizare precisă constituie o secțiune critică a întregului proces. Regiunea determinată de numărul de înmatriculare poate fi localizată folosind diferite metode care pot fi clasificate în funcție de caracteristica utilizată [1]: culoare, textură, sau contur. Ultima categorie este întâlnită cel mai frecvent în aplicații și se bazează pe presupunerea că în interiorul plăcuței există schimbări de intensitate mai drastice și mai dese decât în oricare altă zonă a imaginii. Metodele bazate pe culoare au ca fundație presupunerea că există o distincție clară între culoarea numărului de înmatriculare și fundal. Cele pe textură se folosesc de prezența caracterelor în interiorul plăcuței pentru a delimita regiunea acestuia.

2.0.1 Metode bazate pe caracteristici cromatice

Această categorie de metode vizează obiectele care prezintă în interiorul lor o paletă cromatică bine definită și ușor diferențiabilă de fundalul imaginii. Un exemplu clasic îl reprezintă localizarea semnelor de circulație, a căror culoare standardizată permite separarea lor de restul scenei.

Punctul de plecare al acestor metode îl constituie alegerea unui model croma-

tic. Un model cromatic reprezintă un model matematic abstract care descrie modul în care culorile pot fi reprezentate într-un spațiu cu mai multe dimensiuni, cel mai frecvent tridimensional. Modelul RGB (*Red, Green, Blue*) descrie fiecare culoare prin combinația aditivă a trei componente de bază și corespunde modului în care senzorii camerelor digitale achiziționează imaginea. Dezavantajul său major în contextul localizării este că informația despre culoare și cea despre luminozitate sunt amestecate în cele trei canale: aceeași culoare percepută își modifică semnificativ valorile RGB atunci când condițiile de iluminare se schimbă. Din acest motiv, abordările bazate pe culoare convertește de regulă imaginea într-un model din familia HSV/HSL/HSI (*Hue, Saturation, Value/Lightness/Intensity*), în care nuanța (*hue*) este separată de saturație și de componenta de luminozitate. Această separare conferă o anumită invarianță la variațiile de iluminare, deoarece nuanța unei culori rămâne relativ stabilă chiar și atunci când intensitatea luminii se modifică.

În cadrul sistemelor ALPR, numeroase lucrări au valorificat caracteristicile cromatice ale plăcuței pentru a o localiza [2], [3].

Prima lucrare [2] tratează localizarea numerelor de înmatriculare din China, unde există un număr restrâns de combinații standardizate de culori (de exemplu, caractere albe pe fundal albastru pentru autoturisme sau caractere negre pe fundal galben pentru vehiculele de mare tonaj). Ideea de bază este că o anumită combinație culoare-fundal apare aproape exclusiv în regiunea plăcuței. Trecerea la modelul HLS rezolvă problema robusteții, dar introduce un cost de calcul major: conversia $RGB \rightarrow HLS$ devine punctul critic, necesitând aproximativ o secundă pentru o imagine de 1024×768 pixeli. Soluția finală păstrează insensibilitatea la iluminare a modelului HLS, dar evită conversia propriu-zisă: pixelii sunt clasificați în 13 categorii definite direct în domeniul RGB, în funcție de variația luminozității. Regiunea plăcuței este apoi delimitată prin analiza histogramelor pe direcțiile verticală și orizontală și validată pe baza raportului lățime/înălțime (WHR). Sistemul atinge o rată de citire corectă de 90 % din imagini, în mai puțin de 0,3 secunde. Etapele acestui proces de extracție pot fi urmărite în Figura 2.1.

A doua lucrare [3] abordează plăcuțele din Taiwan, care utilizează patru culori distincte (alb, negru, roșu și verde). De această dată imaginea RGB este transformată efectiv în modelul HSI, pe baza căruia se construiește un detector de contururi cromatice. Spre deosebire de un detector de margini clasic, acesta reacționează doar la cele trei tipuri de tranziții întâlnite în interiorul plăcuței — negru-alb, roșu-alb și verde-alb —, ignorând restul contururilor din imagine. Pentru fiecare dintre hărțile de nuanță, saturație, intensitate și contur se calculează câte o hartă *fuzzy*, ale cărei valori exprimă gradul în care fiecare pixel aparține unei plăcuțe. Cele patru hărți sunt apoi combinate printr-un agregator *fuzzy* în două etape, iar regiunile cu valori maxime locale și dimensiune suficientă sunt reținute drept candidați. Folosind ex-



(a) Localizarea zonei de interes generale folosind proiecții verticale



(b) Extragerea numărului de înmatriculare folosind proiecții verticale



(c) Numărul de înmatriculare extras

Figura 2.1: Procesul de extracție al numărului de înmatriculare [2]

clusiv informația de localizare, metoda atinge o rată de succes de 97,9 %, respectiv 93,7 % pentru întregul sistem de recunoaștere.

Această categorie de metode prezintă atât avantaje, cât și limitări importante. Pe de o parte, atunci când contextul problemei garantează o paletă cromatică standardizată, ele oferă o acuratețe ridicată și un timp de execuție redus, datorită algoritmilor relativ simpli și ușor de optimizat. Pe de altă parte, ele nu generalizează la un spectru larg de formate, fiind dependente de un set restrâns de combinații de culori cunoscute apriori; din acest motiv, lucrările menționate vizează țări precum China sau Taiwan, unde variațiile sunt limitate. În plus, performanța lor rămâne sensibilă la condițiile de iluminare și la degradarea cromatică a plăcuțelor reale.

2.0.2 Metode bazate pe conturul obiectelor

Această abordare are la bază presupunerea că în interiorul plăcuței există o densitate ridicată de linii verticale și orizontale care fac regiunea respectivă să iasă în evidență. Indiferent de abordarea ulterioară, fluxul de procesare începe întotdeauna de la transformarea imaginii în tonuri de gri. În fotografie și viziune pe calcula-

tor, o imagine în tonuri de gri este o imagine în care fiecare pixel este o nuanță de gri, unde valoarea fiecărei celule este un singur eșantion, ținând doar informația intensității. Această caracteristică a imaginilor în tonuri de gri reprezintă un factor important, deoarece în cadrul trecerilor, mai exact la marginile unui obiect apar întotdeauna diferențe mari de intensitate, cu atât mai mult în cadrul unui număr de înmatriculare unde trecerile sunt în general de la alb la negru, roșu la alb, verde la roșu, etc. Treceri bruște, clare și evidente într-o imagine convertită la alb-negru.

În [4] asupra imaginii convertite este aplicat operatorul Sobel pentru a extrage atât marginile verticale cât și cele orizontale. Operatorul Sobel constă în două nuclee specializate pentru detectarea liniilor convoluționate asupra întregii imagini pentru a produce un rezultat care evidențiază marginile obiectelor, trecerile bruște de la o intensitate la alta. Rezultatul intermediar generat de operatorul Sobel este binarizat folosind diferiți algoritmi de prag precum metoda histogramei, *grad4* etc. Pentru o localizare robustă sunt aplicate imaginea *skip* și imaginea integrală, prima având ca scop calcularea, pentru un dreptunghi de dimensiune predefinită, a distanței dintre pixelii albi și cei negri, iar imaginea integrală calculează pentru fiecare punct suma cumulativă a intensităților tuturor pixelilor aflați în regiunea superioară și stângă. Folosind această metodă, autorii au reușit să obțină o rată de localizare cu succes a numărului de înmatriculare de 99,95 % cu un timp mediu de procesare de 47 ms. Rezultatele intermediare ale acestui flux pot fi urmărite în Figura 2.2.

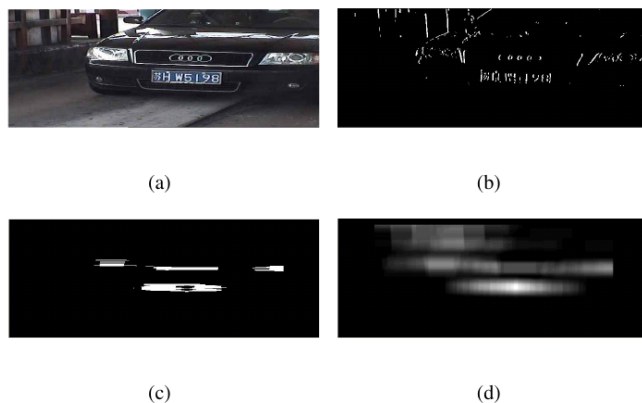


Figura 2.2: (a) Imaginea RGB inițială, (b) Imaginea binară a marginilor, (c) Imaginea *skip*, (d) Imaginea integrală [4]

Similar cu [4], [5] utilizează operatorul Sobel ca componentă primară pentru extragerea marginilor și a conturilor, totuși deosebirile apar încă din etapa de pre-procesare a imaginii. [5] se folosesc de filtrul median pentru a atenua zgomotul imaginii. În viziunea pe calculator, atenuarea zgomotului presupune crearea unei funcții care capturează informațiile și tiparele imaginii lasând la o parte detaliile la scară redusă sau alte variații rapide de intensitate.

Pe lângă filtrarea mediană, pentru a asigura un rezultat robust, autorii definesc o mască de dimensiuni predefinite preluate din condiționarea problemei asupra căreia să continue cu următoarele etape ale procesului, respectiv operatorul Sobel, binarizarea imaginii și operația morfologică de deschidere. Aceasta din urmă constă în aplicarea consecutivă a două transformări fundamentale morfologice: eroziune și dilatare.

Operația de eroziune are rolul de a diminua obiectele din prim-plan și de a elimina structurile izolate care nu aparțin plăcuței. Aceasta presupune aplicarea unui nucleu asupra întregii imagini, pixelul central fiind setat ca alb dacă toți pixelii acoperiți de nucleu sunt la rândul lor albi. Operația de dilatare reprezintă opusul celei de eroziune, este formată din același algoritm, însă structura sa decizională transformă pixelul evaluat în „alb” dacă cel puțin un pixel acoperit de elementul structurant este activ. Rezultatul fiecărei etape a acestui flux de pre-procesare poate fi urmărit în Figura 2.3.

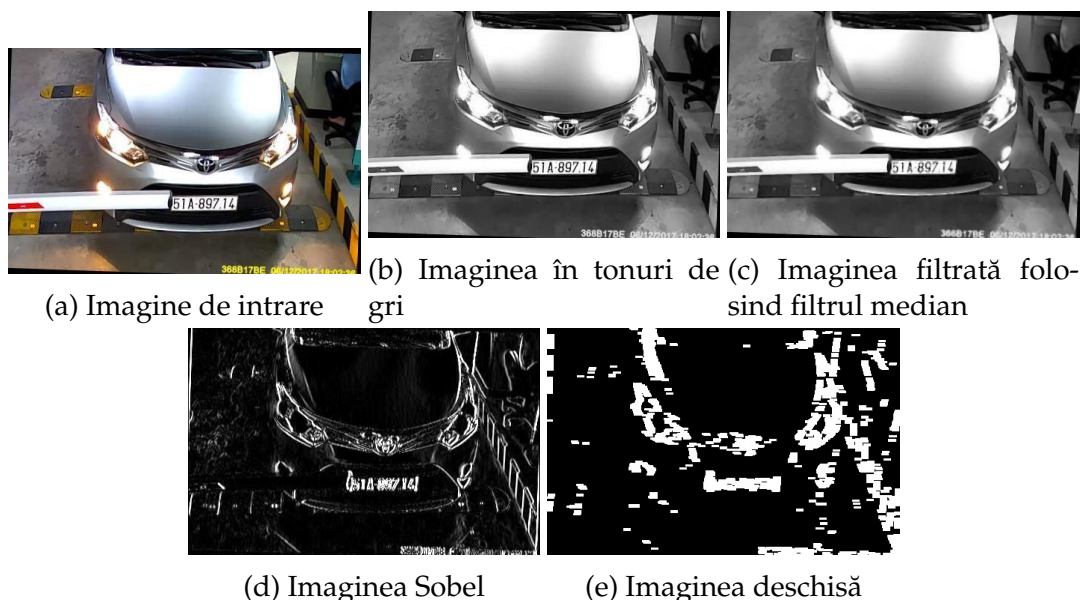


Figura 2.3: Etapele de pre-procesare propuse în [5]: imaginea de intrare, conversia în tonuri de gri, filtrarea mediană, aplicarea operatorului Sobel și operația morfologică de deschidere.

Deși Sobel oferă rezultate bune, costul său din punct de vedere computațional a motivat autorii lucrării [6] să dezvolte un nou algoritm pentru detectarea marginilor verticale. Metoda propusă omite în totalitate marginile orizontale din cadrul unei imagini, axându-se strict pe cele verticale. Această decizie a fost fundamentată pe observația că marginile orizontale pot cauza diferite elemente din compoziția mașinii precum grilajul să fie identificate în mod eronat drept numere de înmatriculare.

Fluxul de procesare propus în [6] diferă față de cele menționate anterior. Inițial,

asupra imaginii în tonuri de gri este aplicat un algoritm de binarizare cu prag adaptiv.

Acesta este urmat de ULEA (*Unwanted Line Elimination Algorithm*), un procedeu specializat de eliminare a artefactelor generate de către binarizare și a liniilor verticale nedorite de o lățime predefinită.

În final este aplicat VEDA, propus ca înlocuitor al operatorului Sobel de către autori. Această metodă constă în convoluționarea unui nucleu de 2×4 asupra întregii imagini și o structură decizională care selectează un pixel drept activ sau inactiv pe baza unor reguli logice în locul operațiilor clasice de convoluție.

Prin renunțarea la procesarea liniilor orizontale și optimizarea mecanismului de detecție, algoritmul propus are o viteză medie de rulare de 15 ms, o îmbunătățire semnificativă în comparație cu operatorul Sobel, care atinge o viteză medie de 130 ms. O comparație vizuală între rezultatele celor doi operatori poate fi urmărită în Figura 2.4.

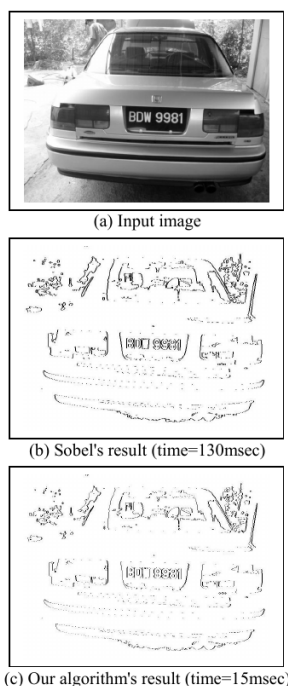


Figura 2.4: Comparație între rezultatele generate de Sobel și VEDA [6]

Datorită faptului că [4], [5], [6] folosesc drept componentă principală operatorul Sobel, respectiv VEDA, aceste abordări sunt capabile să localizeze plăcuțe conținând numere de înmatriculare aflate în imagine la o înclinație de până la 15° . Pentru a combate acest lucru, în [7] este folosit operatorul Hough drept selector principal.

Spre deosebire de operatorii de gradient (Sobel sau VEDA), care se bazează pe analiza locală a pixelilor, transformarea Hough reprezintă o tehnică de extracție a caracteristicilor utilizată pentru a identifica forme geometrice prin procesul de votare într-un spațiu al parametrilor. În contextul localizării numerelor de înmatriculare,

aceasta este utilizată preponderent pentru detecția liniilor drepte care definesc conturul dreptunghiular al plăcuței.

Principiul fundamental constă în maparea fiecărui punct de margine (x, y) din spațiul imaginii în spațiul polar (ρ, θ) , conform relației date de Ecuația 2.1:

$$\rho = x \cos(\theta) + y \sin(\theta), \quad (2.1)$$

unde ρ reprezintă distanța perpendiculară de la origine la dreaptă, iar θ este unghiul dintre axa x și această perpendiculară.

Implementarea transformării Hough în detrimentul implementării operatorului Sobel sau VEDA aduce anumite avantaje din punct de vedere al fiabilității sistemului, acesta fiind capabil să localizeze în mod corect plăcuțe de înmatriculare înclinate la 30° comparat cu 15° . De asemenea, transformarea Hough nu este atât de sensibilă la variațiile de intensitate ale imaginii, astfel că o imagine de calitate nu este imperios necesară.

Totuși, performanța sporită vine cu un cost de complexitate computațională ridicat. Deoarece procesul de votare trebuie efectuat pentru fiecare pixel de margine și pentru o gamă largă de unghiuri, consumul de memorie și timpul de procesare sunt semnificativ mai mari față de un operator de convoluție simplu precum Sobel. Din acest motiv, în sistemele de timp real, transformarea Hough este adesea aplicată pe o regiune de interes (ROI) restrânsă sau este precedată de o etapă de binarizare agresivă pentru a reduce numărul de pixeli procesați.

2.0.3 Metode bazate pe textura obiectelor

Metodele bazate pe textură pornesc de la presupunerea că în interiorul numerelor de înmatriculare prezența caracterelor oferă o textură specifică și ușor diferențiabilă în comparație cu fundalul/ restul elementelor prezente în imagine.

În literatura de specialitate, există multiple abordări ale acestei categorii de metode, [8], [9], care se diferențiază prin modul în care este descrisă textura caracterelor și prin tehnica folosită pentru a evidenția regiunile cu densitate ridicată de tranziții.

În [8] autorii aleg drept caracteristică definitorie a texturii densitatea de linii verticale prezente în cadrul numărului de înmatriculare. Aceștia propun un sistem al cărui prim pas este scăderea cromatică (*color subtraction*), un algoritm ce primește ca parametri de intrare imaginea achiziționată anterior și o listă de elemente dintr-unul dintre modelele cromatice posibile. Acest algoritm va păstra activi din imagine doar pixelii ale căror valori apar în lista primită drept parametru de intrare, iar pe restul îi va dezactiva. Următorul pas este aplicarea operatorului Sobel pentru a evidenția liniile verticale. În continuare va fi construită o hartă de densitate ponderată (*weight-based density map*) folosind algoritmul de asignare a greutăților dat de Ecuația 2.2, în

care creșterea, respectiv descreșterea unei greutatei asignate este direct proporțională:

$$\begin{aligned}
 W_{t_{\text{decr}}} &\propto (i - p) \\
 W_{t_{\text{decr}}} &\propto \gamma \\
 W_{t_{\text{decr}}} &= \partial \gamma (p - i) \\
 W_t &= \sum_{i=0}^n t_w (W_t + (px_i)\gamma + (1 - px_i)(\partial \gamma (p - i))) \\
 t_w &= \frac{W_t - |W_t|}{-2W_t} \beta + \frac{W_t + |W_t|}{2W_t}
 \end{aligned} \tag{2.2}$$

unde ∂ este constanta de decădere (*decay constant*), γ este factorul de incrementare a greutății, β greutatea inițială, n numărul de puncte de coordonate, i coordonata muchiei curente, p coordonata muchiei anterioare, iar W_t valoarea greutății.

După finalizarea construcției hărții de densitate putem identifica posibilele regiuni în care apare un număr de înmatriculare.

Spre deosebire de [8], care descrie textura prin densitatea liniilor verticale, în [9] autorii pornesc de la ideea că plăcuța poate fi privită ca o regiune în care textura imaginii prezintă o „neregularitate” locală pronunțată, mai exact treceri bruște și dese între caracterele întunecate și fundalul deschis. Pentru a evidenția aceste zone, ei propun o tehnică de segmentare adaptivă numită SCW (*Sliding Concentric Windows*), care nu măsoară direct contururile, ci variația statistică a intensităților dintr-o vecinătate.

Metoda se bazează pe utilizarea a două ferestre dreptunghiulare concentrice, una interioară A de dimensiune $(2X_1) \times (2Y_1)$ și una exterioară B de dimensiune $(2X_2) \times (2Y_2)$, care parcurg împreună întreaga imagine, pixel cu pixel, pornind din colțul stânga-sus. Pentru fiecare poziție se calculează în cele două ferestre câte o măsură statistică M (media sau deviația standard a intensităților), iar pixelul central este clasificat în funcție de raportul dintre cele două măsuri, după cum este descris în Ecuația 2.3:

$$I_{\text{AND}}(x, y) = \begin{cases} 0, & \text{dacă } \frac{M_B}{M_A} \leq T \\ 1, & \text{dacă } \frac{M_B}{M_A} > T \end{cases} \tag{2.3}$$

unde T este un prag stabilit experimental. Atunci când raportul depășește pragul, vecinătatea pixelului prezintă o variație texturală suficient de mare pentru ca acesta să fie considerat parte a unei regiuni de interes. Parametrii X_1, X_2, Y_1, Y_2 se aleg astfel încât raportul de aspect al ferestrelor să fie apropiat de cel al obiectelor căutate (caracterele plăcuței).

Imaginea binară I_{AND} obținută prin SCW este apoi combinată cu imaginea originală printr-o operație logică ȘI, rezultatul fiind binarizat cu ajutorul metodei de prag

adaptiv Sauvola, în care pragul fiecărui pixel depinde de media și deviația standard locale dintr-o fereastră de dimensiune $b \times b$, conform Ecuației 2.4:

$$T(x, y) = m(x, y) \left[1 + k \left(\frac{\sigma(x, y)}{R} - 1 \right) \right]. \quad (2.4)$$

Asupra imaginii binarizate se aplică o analiză a componentelor conexe (CCA), iar obiectele rezultate sunt filtrate pe baza unor criterii geometrice: raportul de aspect, orientarea (sub 35°) și numărul Euler. Acesta din urmă, definit ca diferența dintre numărul de obiecte și numărul de „găuri” interioare, este folosit pe baza observației că o plăcuță validă conține cel puțin patru caractere, deci minimum trei goluri ($E > 3$); regiunile care nu îndeplinesc aceste condiții sunt eliminate. Pentru a trata și plăcuțele cu fundal închis și caractere deschise, dacă nicio regiune candidată nu este găsită, imaginea este inversată și întregul proces reia.

Etapa de localizare este urmată de segmentarea caracterelor, realizată tot prin SCW aplicat regiunii decupate, și de recunoașterea propriu-zisă cu o rețea neuronală probabilistică (PNN) cu topologia 108-180-36. Testat pe un set de 1334 de imagini din scene naturale, cu fundaluri și condiții de iluminare variate, sistemul a segmentat corect plăcuța în 96,5 % din cazuri, recunoașterea întregii plăcuțe atingând 89,1 %, respectiv o rată globală de succes de 86 %.

Capitolul 3

Segmentarea imaginii

Segmentarea imaginii joacă un rol crucial în sistemele ALPR. Procesul presupune divizarea imaginii într-un anumit număr de segmente informative, sau regiuni care sunt mai simple de studiat.

În literatura de specialitate, pentru a segmenta numărul de înmatriculare sunt propuse mai multe tehnici [1]:

- Analiza conectivității pixelilor [10];
- Analiza proiecțiilor [7];
- Utilizarea informațiilor apriori privind aspectul caracterelor [11];
- Analiza conturului caracterelor [12]

3.0.1 Conectivitatea pixelilor

Această abordare presupune binarizarea regiunii de interes extrase de modulul anterior, urmată de analiza componentelor conexe (CCA) și de aplicarea unei structuri decizionale pentru filtrarea regiunilor identificate de CCA. Selecția se bazează în principal pe criterii precum similitudinea dimensiunilor și a proporțiilor zonelor.

Principalul dezavantaj al acestei metode este sensibilitatea la rotație, selecția fiind realizată pe baza uniformității geometrice a regiunilor, numerele de înmatriculare rotite sau înclinate parțial sunt adesea segmentate în mod incorect.

3.0.2 Analiza proiecțiilor

Contrastul ridicat dintre fundalul plăcuței și caractere face ca, în urma binarizării, cele două să fie reprezentate prin valori opuse. Această proprietate stă la baza unei familii largi de metode de segmentare, care exploatează distribuția pixelilor prin combinarea analizei proiecțiilor verticale și orizontale.

O proiecție se obține prin construirea unei histograme a densității pixelilor activi de-a lungul uneia dintre cele două axe de coordonate, un pixel fiind considerat activ atunci când valoarea sa aparține unui interval predefinit. În cazul, frecvent întâlnit în literatura de specialitate, al imaginilor binarizate, pixelii activi iau una dintre valorile 0 sau 255.

Un exemplu reprezentativ este sistemul propus în [7], în care extracția numărului de înmatriculare se bazează pe transformarea Hough pentru corectarea variațiilor de înclinare și rotație.

Segmentarea începe cu analiza proiecțiilor orizontale, prin care numărul de înmatriculare este divizat pe rânduri. Etapa este esențială pentru plăcuțele dispuse pe mai multe rânduri, întrucât separarea pe rânduri normalizează proporțiile și dimensiunea relativă a caracterelor în raport cu sub-segmentul care le conține. În absența ei, o evaluare globală ar introduce discrepante majore: pe un singur rând un caracter ocupă aproximativ 85% din înălțimea imaginii, în timp ce pe o plăcuță cu două rânduri ajunge la cel mult 50%.

După divizarea pe rânduri, proiecțiile verticale sunt utilizate pentru a determina pozițiile de început și de final ale fiecărui caracter. Segmentele astfel obținute sunt apoi filtrate printr-un set de constrângeri, menite să elimine elementele segmentate eronat drept caractere.

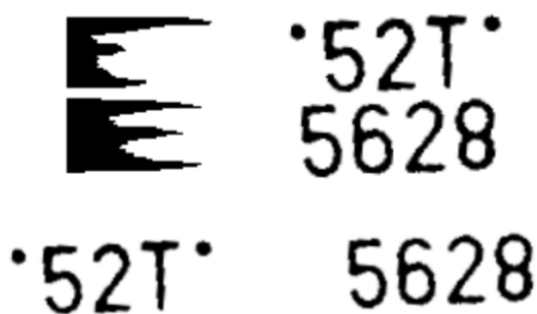


Figura 3.1: Rezultatul divizării pe rânduri [7]

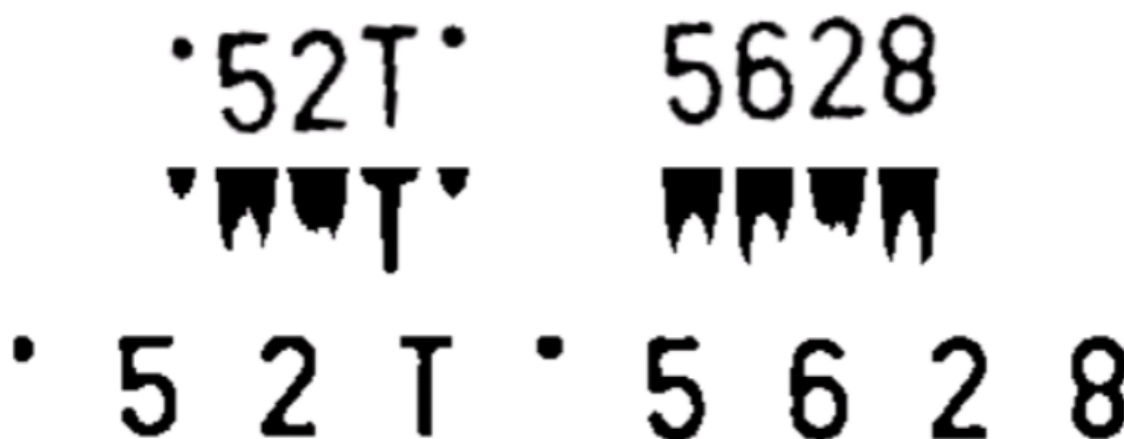


Figura 3.2: Rezultatul segmentării folosind proiecții verticale [7]

3.0.3 Utilizarea informațiilor apriori privind aspectul caracterelor

Cunoștințe apriori în legătură cu aspectul caracterelor, font-ul, mărimea și proporțiile lor pot îmbunătăți în mod semnificativ procesul de segmentare.

În [11] este propusă o abordare similară cu cea prezentată anterior în [7] în care numărul de înmatriculare este adus la o mărime și înclinare standard, iar etapa de segmentare este realizată folosind proiecții orizontale și verticale. Spre deosebire de metoda amintită în mod anterior, autorii se folosesc pe parcursul etapei de segmentare de o structură decizională care va ajuta la eliminarea rezultatelor fals-pozitive.

Particularitatea acestei abordări constă în combinarea informației furnizate de proiecția verticală cu cea obținută printr-o etapă preliminară de analiză a componentelor conexe (pre-extragerea zonelor conexe). Rolul acestei etape preliminare este de a atenua efectul zgomotului asupra proiecției: în prezența zgomotului, profilul de proiecție al imaginii binarizate prezintă punți (conexiuni) între caractere, care îngreunează separarea lor. Prin filtrarea prealabilă a zonelor conexe, numărul acestor conexiuni din profilul de proiecție se reduce semnificativ, motiv pentru care varianta astfel curățată este reținută pentru pașii următori, în detrimentul proiecției obținute direct din imaginea zgomotoasă.

Pe lângă atenuarea zgomotului, zonele conexe pre-extrase furnizează și informațiile apriori care stau la baza extragerii: lățimea caracterelor și poziția lor în cadrul plăcuței. Conform specificului plăcuțelor de înmatriculare avute în vedere, toate caracterele au aceeași lățime, care poate fi estimată direct din lățimea unei zone conexe deja izolate corect (de exemplu cifrele 5 sau 0). Se definește poziția centrală a unui caracter ca fiind poziția orizontală a marginii sale stângi la care se adaugă jumătate din lățimea caracterului, iar distanța dintre pozițiile centrale a două caractere vecine se numește distanță între pozițiile centrale.

Geometria plăcuței impune o constrângere apriori suplimentară asupra acestor distanțe: ultimele cinci caractere au aceeași distanță între pozițiile centrale, în timp ce distanța d_0 dintre al doilea și al treilea caracter este mai mare decât distanța d_1 dintre celelalte caractere ($d_0 > d_1$). Experimental, cele două mărimi se află în relația dată de Ecuația 3.1:

$$d_0 = \lambda \cdot d_1, \quad \lambda \in [1.2, 1.3]. \quad (3.1)$$

Folosind aceste cunoștințe apriori, procesul de extragere a caracterelor parcurge următorii pași:

1. Din zona conexă pre-extrasă se determină intervalul vertical h_0 în care se află caracterele plăcuței.
2. Tot din zona conexă pre-extrasă se obțin lățimea caracterului w_1 și pozițiile centrale ale caracterelor, ținând cont de constrângerile geometrice de mai sus (lățime constantă și distanțele d_0, d_1 dintre pozițiile centrale).
3. Se scanează zonele conexe cuprinse în intervalul h_0 din imaginea plăcuței, integrând informația de poziție furnizată de proiecție; în funcție de rezultat se realizează fie combinarea mai multor zone într-un singur caracter, fie divizarea unei zone care înglobează mai multe caractere.
4. Se furnizează rezultatul extragerii, care conține pozițiile caracterelor, acestea putând fi astfel izolate cu precizie.

Acest proces adițional de selecție a regiunilor candidat sporește întreaga fiabilitate a sistemului deoarece elementele plăcuței care nu aparțin în-sine de numărul de înmatriculare precum șuruburile de fixare extrase în fig. 3.2 nu vor fi considerate mai departe drept zone de interes.

3.0.4 Analiza conturului caracterelor

O altă abordare are la bază detecția conturilor caracterelor. Pentru aceasta, în [12] este propus un proces compus din doi pași, fundamentat pe tehnici de segmentare ce utilizează metoda *Fast Marching* ghidată de formă (shape-driven).

Metoda *Fast Marching*, introdusă de Sethian [13], este un algoritm numeric folosit pentru a urmări evoluția unui front (a unei curbe) care se propagă monoton într-o singură direcție, dinspre o regiune inițială către exterior. Algoritmul este o particularizare a metodelor de tip *level-set*, în care viteza de propagare este strict pozitivă, astfel încât odată ce frontul a depășit un punct nu se mai poate întoarce.

Din punct de vedere matematic, metoda calculează, pentru fiecare punct x al imaginii, momentul de timp $T(x)$ la care frontul ajunge în acel punct. Această

mărimă este soluția ecuației eikonale, dată de Ecuația 3.2:

$$|\nabla T(x)| F(x) = 1, \quad (3.2)$$

unde $F(x)$ reprezintă funcția de viteză, care determină cât de rapid avansează frontul în fiecare punct. În aplicațiile de segmentare, funcția de viteză este derivată din informația din imagine (de regulă din mărimea gradientului): frontul se deplasează rapid în zonele omogene și încetinește, oprindu-se, în apropierea muchiilor, unde gradientul este mare. În acest fel, contururile obiectelor sunt delimitate în mod natural de zonele în care frontul stagnează.

Particularitatea abordării din [12] constă în ghidarea propagării frontului nu doar de informația de contur, ci și de modele statistice ale formei caracterelor (*shape-driven*). Funcția de viteză este astfel construită încât să integreze cunoștințe a priori despre forma caracterelor, permițând curbei să se oprească în dreptul muchiilor reale ale obiectelor și, în același timp, să inspecteze gradul de similaritate dintre conturul obținut și formele caracterelor cunoscute. Procesul realizează concomitent segmentarea și recunoașterea: pe parcursul evoluției frontului sunt determinate atât pozițiile delimitatoare ale fiecărui caracter, cât și eticheta caracterului corespunzător, eliminând necesitatea unei etape separate de recunoaștere.

Capitolul 4

Recunoașterea caracterelor

Optofonul, dezvoltat în anul 1914, a fost unul dintre primele aparate electronice destinate citirii automate a textului tipărit. Funcționarea sa se baza pe proprietatea seleniului de a conduce curentul electric în mod diferit în funcție de intensitatea luminii: pe măsură ce dispozitivul scana o pagină, distingea zonele întunecate, corespunzătoare textului, de spațiile albe dintre cuvinte.

De la optofon și până astăzi, domeniul recunoașterii automate a caracterelor (OCR) a cunoscut progrese remarcabile, evoluând de la simpla detecție a contrastului către modele capabile să interpreteze caractere în condiții reale, cu variații de font, dimensiune și iluminare. În cele ce urmează sunt prezentate fundamentele teoretice ale celor mai importante abordări, de la metoda clasică a potrivirii de șabloane până la clasificatorii statistici, cu accent pe aplicarea acestora în recunoașterea numerelor de înmatriculare.

4.1 Potrivire de șabloane

Această abordare, cunoscută sub denumirea de *template matching*, reprezintă metoda clasică de recunoaștere a caracterelor prin compararea directă a pixelilor. Procesul presupune existența unei baze de date cu șabloane (matrițe) predefinite pentru fiecare caracter, care sunt comparate succesiv cu regiunea segmentată până la identificarea celei mai bune potriviri.

Deși este eficientă din punct de vedere computațional datorită simplității algoritmice, această tehnică prezintă limitări severe în scenarii reale. Ea este aplicabilă cu succes doar atunci când caracterele extrase au dimensiuni, fonturi și grosimi constante, fiind extrem de sensibilă la variații geometrice.

Pentru a introduce un grad de robustețe și pentru a atenua sensibilitatea la zgomot sau mici distorsiuni, în locul unei suprapuneri binare perfecte, se utilizează al-

goritmi bazați pe calcularea distanțelor matematice între vectorii de trăsături: distanța Hamming [14], valoarea Jaccard [15].

În mod concret, [14] prezintă un sistem de recunoaștere a caracterelor format din două etape: normalizarea caracterelor și potrivirea șabloanelor. Prima etapă presupune aducerea regiunilor de interes extrase în etapa anterioară la o dimensiune standard de 40×40 prin intermediul unei transformări afine care mapează pixelii din imaginea originală la cei ai regiunii determinate anterior. Efectul acestei normalizări poate fi urmărit în Figura 4.1.

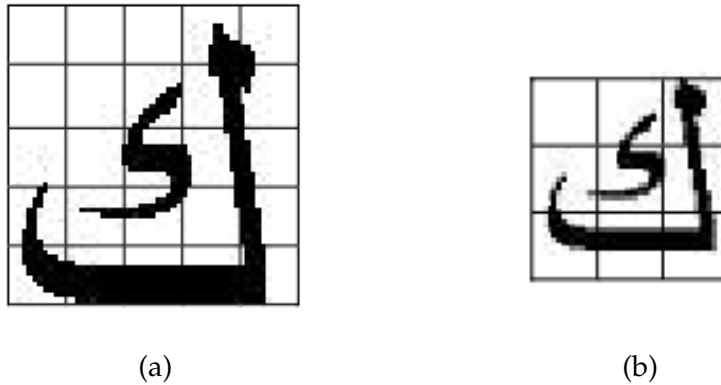


Figura 4.1: (a) Caracterul original, (b) Caracterul normalizat la o dimensiune standard de 40×40 [14].

Pentru etapa de potrivire a șabloanelor, precum am menționat anterior, autorii se folosesc de distanța Hamming pentru a determina cel mai apropiat caracter. Formula acestei distanțe este descrisă de Ecuația 4.1.

$$\sum_{i=1}^{nrows} \sum_{j=1}^{ncols} \text{mismatch}_{i,j} \quad (4.1)$$

$$\text{mismatch}_{i,j} = \begin{cases} 1, & \text{if } \text{original}_{i,j} \neq \text{extracted}_{i,j} \\ 0, & \text{if } \text{original}_{i,j} = \text{extracted}_{i,j} \end{cases}$$

Unde $nrows$ și $ncols$ sunt numărul de linii respectiv cel de coloane.

4.2 Clasificatori statistici

Această metodă valorifică avansurile recente din domeniul inteligenței artificiale pentru a crea module de recunoaștere a caracterelor robuste. Acestea prezintă o rezistență ridicată la variațiile de luminozitate, dimensiuni sau fonturi variate utilizate în creare numerelor de înmatriculare, etc. În literatura de specialitate, cele mai

frecvent utilizate modele pentru această etapă a procesării imaginii sunt rețelele neuronale artificiale (ANN) și mașinile pe suport vectorial (SVM), ambele prezentând avantaje și limitări specifice.

S-ar putea spune că există o oarecare similitudine între SVM și ANN deoarece ambele își au rădăcinile în algoritmul perceptronului [16], însă abordările matematice care succedă acest algoritm diferă fundamental.

4.2.1 Mașini pe suport vectorial

Algoritmul ce stă la baza mașinilor pe suport vectorial reprezintă o generalizare a algoritmului "portret generalizat" dezvoltat la începutul anilor 1960 [17], [18]. Fundamentele matematice ale regresiei/ clasificării vectoriale sunt ancorate în învățarea statistică și teoria VC, care oferă o măsură a capacității de învățare a unui algoritm [19].

Este important de menționat că algoritmul perceptronului funcționează prin actualizarea vectorului de greutate $\mathbf{w}$ pornind de la o valoare inițială arbitrară, de regulă vectorul nul. Acesta este modificat prin adunarea exemplelor pozitive clasificate în mod eronat ca fiind negative, sau prin scăderea exemplelor negative clasificate ca fiind pozitive. Așadar, rezultatul final va fi întotdeauna o combinație liniară a punctelor din setul de antrenament, după cum este exprimat în Ecuația 4.2:

$$\mathbf{w} = \sum_{i=1}^{\ell} \alpha_i y_i \mathbf{x}_i, \quad (4.2)$$

unde semnul ecuației este dat de clasificarea $y_i \in \{-1, 1\}$, iar valorile α_i sunt valori pozitive proporționale cu numărul de misclasificări făcute asupra punctului în timpul stagiului de antrenament. Punctele care au cauzat mai puține erori, vor avea un α_i mai mic, iar punctele dificile unul mai mare. Date fiind aceste valori, perceptronul poate fi rescris sub forma dată de Ecuația 4.3:

$$h(\mathbf{x}) = \text{sgn} \left(\sum_{j=1}^{\ell} \alpha_j y_j \langle \mathbf{x}_j \cdot \mathbf{x} \rangle + b \right), \quad (4.3)$$

numită și forma duală a perceptronului.

Învățarea în spațiul trăsăturilor

Complexitatea funcției care trebuie aproximată de modelul de învățare depinde în mod direct de modul în care sunt reprezentate datele de intrare; astfel, dificultatea procesului de învățare poate varia semnificativ în funcție de această reprezentare. În mod ideal, se urmărește alegerea unei reprezentări adaptate specificului problemei

de învățare. O metodă frecvent utilizată în acest scop o constituie transformarea reprezentării datelor printr-o funcție de mapare a trăsăturilor, care proiectează fiecare punct din spațiul de intrare într-un nou spațiu, denumit spațiul trăsăturilor, conform Ecuației 4.4:

$$x = (x_1, x_2, \dots, x_n) \mapsto \phi(x) = (\phi_1(x), \phi_2(x), \dots, \phi_N(x)). \quad (4.4)$$

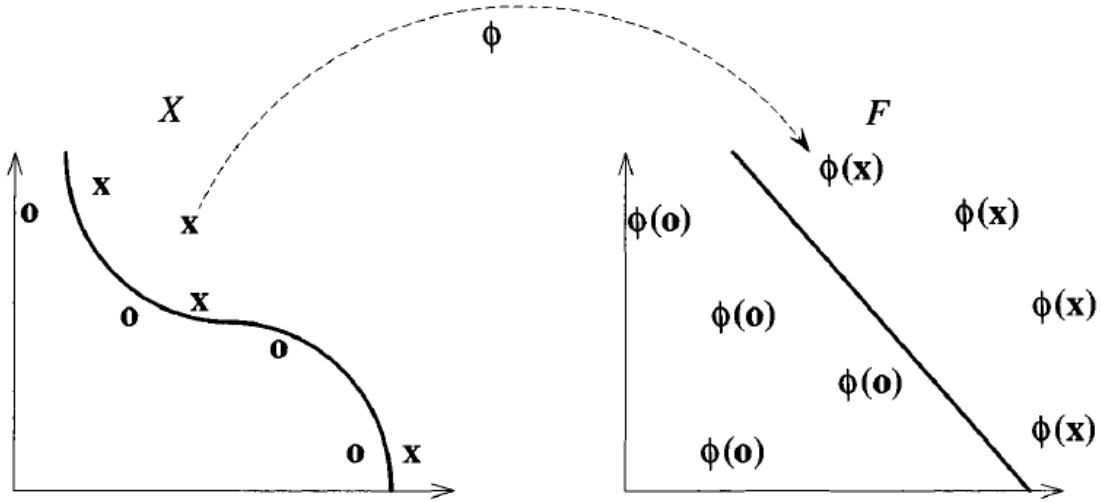


Figura 4.2: O funcție de mapare a trăsăturilor poate simplifica problema de clasificare, transformând un set neliniar separabil într-unul liniar separabil [20].

După cum se poate urmări în Figura 4.2, o astfel de mapare poate transforma un set de date neliniar separabil într-unul liniar separabil. Datorită varietății mari a problemelor de învățare, spațiul trăsăturilor rezultat are, de regulă, o dimensiune mult mai mare decât spațiul de intrare (uneori chiar infinită). În consecință, maparea explicită a fiecărui punct în acest spațiu și rezolvarea directă a problemei de învățare, de forma dată de Ecuația 4.5:

$$f(x) = \sum_{i=1}^N w_i \phi_i(x) + b, \quad (4.5)$$

devin nefezabile din punct de vedere computațional, întrucât ar necesita calcularea și stocarea explicită a tuturor celor N componente $\phi_i(x)$.

Reprezentarea duală introdusă anterior permite însă rescrierea problemei de învățare în spațiul trăsăturilor sub forma dată de Ecuația 4.6:

$$f(x) = \sum_{i=1}^{\ell} \alpha_i y_i \langle \phi(x_i), \phi(x) \rangle + b. \quad (4.6)$$

În această formulare, datele de intrare apar exclusiv prin intermediul produsului scalar $\langle \phi(x_i), \phi(x) \rangle$ din spațiul trăsăturilor. Prin urmare, dacă acest produs scalar ar putea fi calculat direct, ca o funcție definită pe punctele din spațiul de intrare, fără a construi explicit maparea ϕ , atunci cei doi pași (maparea datelor și rezolvarea problemei de învățare) ar putea fi combinați, obținând mașini de învățare capabile să separe date care nu sunt liniar separabile în spațiul original.

O astfel de funcție de mapare implicită din spațiul de intrare în cel al trăsăturilor se numește funcție nucleu (*kernel*) și stă la baza dezvoltării mașinilor pe suport vectorial, pas numit în literatura de specialitate *kernel trick*.

Teoria VC și clasificatori cu marjă maximală

Maparea implicită în spațiul trăsăturilor, realizată prin intermediul funcțiilor nucleu, permite mașinilor pe suport vectorial să definească soluții în spații de dimensiune foarte mare, eventual chiar infinită. Această flexibilitate constituie însă o sabie cu două tăișuri: cu cât familia de funcții pe care modelul o poate reprezenta este mai bogată, cu atât crește și riscul ca acesta să se muleze perfect pe setul de antrenament memorându-l în întregime fără a surprinde structura reală a datelor și, prin urmare, fără a generaliza la exemple noi. Se ridică astfel problema fundamentală a cuantificării capacității unui model, cu scopul de a controla echilibrul dintre fidelitatea față de datele de antrenament și capacitatea de generalizare.

Această problemă este abordată de teoria Vapnik-Chervonenkis (VC), dezvoltată în strânsă legătură cu cadrul învățării probabil aproximativ corecte (*Probably Approximately Correct*, PAC) [21], care formalizează chiar noțiunea de învățare cu garanții asupra generalizării. Tocmai pentru că oferă instrumentele necesare controlului capacității unui model, teoria VC reprezintă fundamentul teoretic indispensabil al mașinilor pe suport vectorial.

Conceptul central al acestei teorii îl constituie dimensiunea VC, o măsură a capacității de exprimare a unei familii de funcții de clasificare. În mod formal, dimensiunea VC reprezintă cardinalitatea celui mai mare set de puncte care poate fi etichetat (pulverizat) în toate modurile binare posibile de către funcțiile clasei respective. O dimensiune VC ridicată indică un model cu o capacitate sporită de a se mula pe orice configurație a datelor, deoarece marja garantată între eroarea empirică (pe setul de antrenament) și eroarea reală (așteptată pe distribuția datelor) crește odată cu dimensiunea VC. În consecință, controlul acestei dimensiuni reprezintă o cerință esențială pentru asigurarea capacității de generalizare a modelului.

Pe baza acestei mărimi, teoria VC stabilește o legătură cantitativă între eroarea măsurată pe setul de antrenament și eroarea așteptată pe întreaga distribuție a datelor. Astfel, pentru orice ipoteză h aparținând clasei de funcții analizate, capacitatea

de generalizare poate fi mărginită superior, cu o probabilitate de cel puțin $1 - \delta$, prin inegalitatea dată de Ecuația 4.7:

$$err_D(h) \leq \epsilon(l, H, \delta) = \frac{2k}{l} + \frac{4}{l} \left(d \log \frac{2el}{d} + \log \frac{4}{\delta} \right), \quad (4.7)$$

unde semnificația fiecărui termen este următoarea:

- $err_D(h)$ — eroarea reală (de generalizare) a ipotezei h , definită drept probabilitatea ca aceasta să clasifice greșit un exemplu extras aleator din distribuția D a datelor;
- $\epsilon(l, H, \delta)$ — marginea superioară garantată asupra erorii de generalizare, exprimată în funcție de dimensiunea setului de antrenament, de complexitatea clasei de funcții și de nivelul de încredere impus;
- l — numărul de exemple din setul de antrenament; pe măsură ce l crește, marginea descrește, reflectând faptul că un volum mai mare de date conduce la garanții mai stricte asupra generalizării;
- H — clasa de funcții de clasificare (ipoteze) din care este selectată soluția;
- d — dimensiunea VC a clasei H , care cuantifică capacitatea de exprimare a acesteia; o valoare mare a lui d mărește marginea, semnalând un risc sporit de supra-învățare;
- k — numărul de exemple clasificate eronat pe setul de antrenament (eroarea empirică), care exprimă fidelitatea ipotezei față de datele observate;
- δ — nivelul de încredere: inegalitatea este satisfăcută cu o probabilitate de cel puțin $1 - \delta$, astfel încât δ reprezintă probabilitatea (mică) ca marginea să nu fie respectată;
- e — baza logaritmului natural (constanta lui Euler).

Structura acestei margini evidențiază compromisul fundamental dintre fidelitatea față de datele de antrenament și capacitatea de generalizare: primul termen, $2k/l$, penalizează erorile empirice, în timp ce al doilea termen crește odată cu dimensiunea VC d , penalizând astfel complexitatea excesivă a clasei de funcții.

În mod concret, mașinile pe suport vectorial restrâng familia clasificatorilor și dimensiunea VC a acestora prin adăugarea următoarei constrângeri: hiperplanul care separă elementele din setul de antrenament trebuie să fie unul cu marja maximă. Adăugând această constrângere și presupunând că punctele de antrenament sunt incluse într-o sferă de rază R , atunci dimensiunea VC efectivă a clasei rezultate

este mărginită superior de raportul R^2/γ^2 [20], unde γ reprezintă marja geometrică a hiperplanului separator.

Formal, problema clasificatorului cu marjă maximală se reduce la rezolvarea problemei de optimizare convexă dată de Ecuația 4.8:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{cu restricția} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, \ell, \quad (4.8)$$

unde minimizarea normei $\|\mathbf{w}\|$ este echivalentă cu maximizarea marjei $\gamma = 1/\|\mathbf{w}\|$. Această proprietate este deosebit de relevantă în contextul recunoașterii caracterelor de pe plăcuțele de înmatriculare, deoarece spațiile de trăsături utilizate (vectori obținuți prin caracteristici HOG, momente Zernike sau pixeli normalizați) sunt frecvent înalt dimensionale, iar capacitatea unui clasificator de a evita supra-învățarea într-un astfel de regim este critică.

Pentru a permite tratarea seturilor de date care nu sunt liniar separabile, formularea originală a fost extinsă prin introducerea variabilelor de relaxare ξ_i și a parametrului de penalizare C [22], conducând la formularea cu marjă moale, dată de Ecuația 4.9:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^{\ell} \xi_i \quad \text{cu restricția} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \quad (4.9)$$

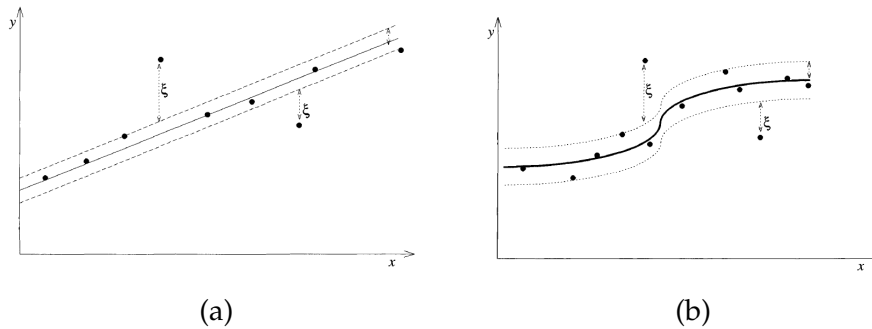


Figura 4.3: (a) Variabila de relaxare pentru exemple liniar separabile, (b) Variabila de relaxare pentru exemple non-liniar separabile

Rolul variabilelor de relaxare ξ_i în cele două situații poate fi urmărit în Figura 4.3. Parametrul C controlează raportul dintre maximizarea marjei și tolerarea erorilor de clasificare, permițând astfel un compromis explicit între capacitatea modelului și riscul empiric. Combinat cu utilizarea funcțiilor kernel introduse prin [23], acest cadru permite construirea unor clasificatori cu putere de discriminare ridicată, păstrând în același timp un control formal asupra capacității de generalizare.

4.2.2 Rețele neuronale artificiale

Rețelele neuronale artificiale (ANN) reprezintă cea de-a doua direcție majoră în recunoașterea caracterelor din cadrul plăcuțelor de înmatriculare. Spre deosebire de SVM, care își are fundamentul în teoria învățării statistice, ANN-urile sunt inspirate dintr-un model simplificat al activității neuronilor biologici.

Această abordare inspirată din anatomia umană nu poate părea evidentă în mod inițial, însă de-a lungul istoriei evoluția a conferit creierului uman o multitudine de caracteristici de dorit care nu sunt prezente într-o mașină von Neumann sau o mașină de calcul modernă:

- Paralelism uriaș
- Capacitatea de învățare
- Capacitatea de generalizare
- Adaptabilitate

Arhitectura perceptronului multistrat

Din punct de vedere structural, o rețea neuronală artificială poate fi modelată ca un graf orientat și ponderat, în care nodurile corespund neuronilor artificiali, iar muchiile descriu conexiunile prin care ieșirea unui neuron devine intrare pentru un altul. Fiecărei conexiuni îi este asociată o pondere ce reglează intensitatea semnalului transmis, ponderi care sunt ajustate pe parcursul procesului de învățare.

Perceptronul multistrat (MLP) este alcătuit dintr-un strat de intrare, unul sau mai multe straturi ascunse și un strat de ieșire; adoptând convenția uzuală conform căreia nodurile de intrare nu sunt considerate un strat propriu-zis. Fiecare neuron j dintr-un strat efectuează o transformare afină asupra activărilor stratului precedent, urmată de aplicarea unei funcții de activare neliniare σ , conform Ecuației 4.10:

$$a_j = \sigma \left(\sum_i w_{ji} x_i + b_j \right). \quad (4.10)$$

O interpretare geometrică intuitivă a acestei arhitecturi evidențiază rolul fiecărui strat în construirea frontierei de decizie: fiecare unitate din primul strat ascuns definește un hiperplan care separă spațiul de intrare, neuronii celui de-al doilea strat combină aceste hiperplane în regiuni de decizie, iar stratul de ieșire le reunește, printr-o operație de tip conjuncție (AND) a separărilor obținute, într-o regiune de decizie finală de formă arbitrară. Această interpretare geometrică poate fi urmărită în Figura 4.4.

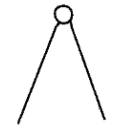
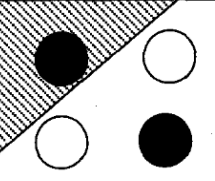
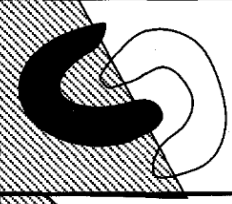
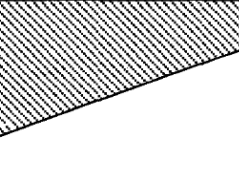
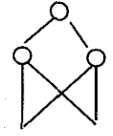
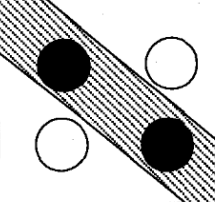
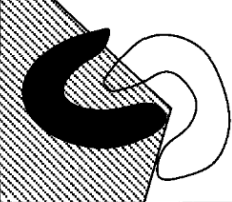
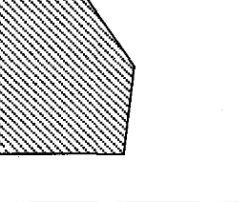
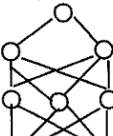
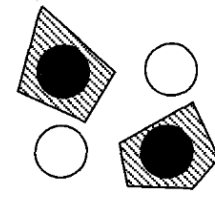
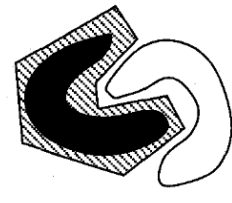
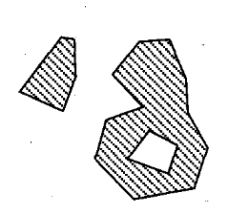
Structure	Description of decision regions	Exclusive-OR problem	Classes with meshed regions	General region shapes
 Single layer	Half plane bounded by hyperplane			
 Two layer	Arbitrary (complexity limited by number of hidden units)			
 Three layer	Arbitrary (complexity limited by number of hidden units)			

Figura 4.4: O interpretare geometrică a rolului straturile ascunse [24]

Compoziția acestor transformări permite rețelei să construiască, în mod implicit, reprezentări ale datelor de complexitate crescândă. Justificarea teoretică pentru această abordare este oferită de teorema aproximării universale, care stabilește că o rețea cu un singur strat ascuns și un număr suficient de neuroni poate aproxima, cu precizie arbitrară, orice funcție continuă definită pe un domeniu compact [25]. Arhitectura generală a unei rețele de tip MLP cu trei straturi poate fi urmărită în Figura 4.5.

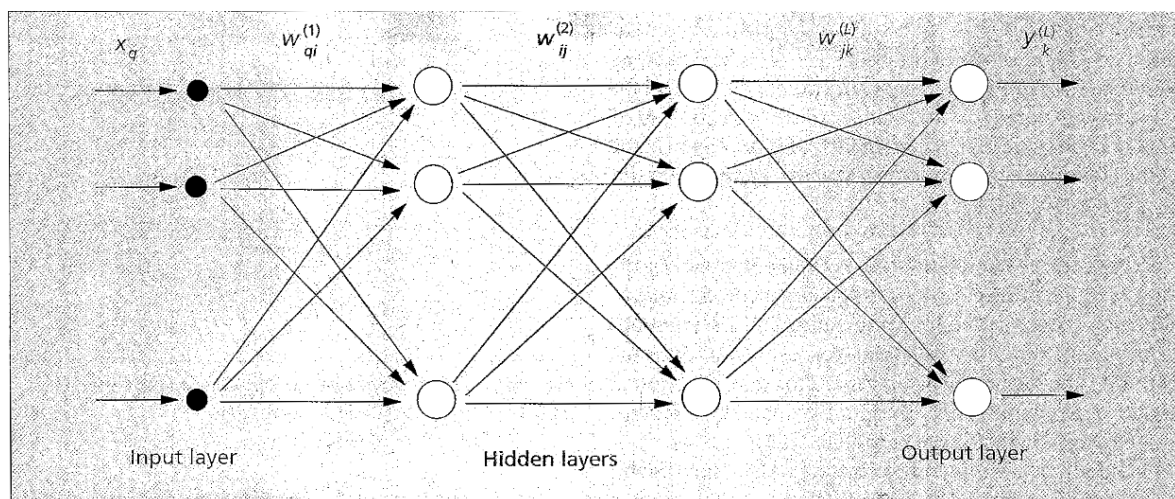


Figura 4.5: Arhitectura unei rețele de tip MLP cu 3 straturi [24]

Antrenarea prin retropropagarea erorii

Spre deosebire de mașinile pe suport vectorial, a căror antrenare se reduce la o problemă de optimizare convexă cu soluție unică, antrenarea unei rețele neuronale presupune minimizarea unei funcții de eroare puternic neliniare în raport cu ponderile, ceea ce conduce, în general, la o suprafață de eroare cu numeroase minime locale.

Procesul de învățare pornește de la definirea unei funcții de cost care cuantifică discrepanța dintre ieșirile produse de rețea și valorile țintă asociate exemplelor de antrenament. O alegere uzuală o constituie eroarea pătratică, dată de Ecuația 4.11:

$$E = \frac{1}{2} \sum_k (t_k - y_k)^2, \quad (4.11)$$

unde t_k reprezintă valoarea dorită, iar y_k ieșirea efectivă a neuronului k din stratul de ieșire. Antrenarea revine astfel la ajustarea iterativă a ponderilor în direcția care reduce această eroare, folosind metoda coborârii gradientului: fiecare pondere este modificată proporțional cu gradientul negativ al erorii în raport cu ea, după cum este descris în Ecuația 4.12:

$$w_{ji} \leftarrow w_{ji} - \eta \frac{\partial E}{\partial w_{ji}}, \quad (4.12)$$

unde η este rata de învățare, un hiperparametru care controlează amplitudinea pasului de actualizare.

Dificultatea principală constă în calcularea derivatelor parțiale $\partial E / \partial w_{ji}$ pentru ponderile situate în straturile ascunse, unde contribuția la eroarea finală nu este una directă. Algoritmul de retropropagare a erorii [26] rezolvă această problemă aplicând în mod sistematic regula de derivare în lanț, distribuind eroarea de la ieșire înapoi către straturile anterioare. El se desfășoară în două etape:

- o etapă de propagare înainte (*forward pass*), în care exemplul de intrare traversează rețeaua strat cu strat, producând activările tuturor neuronilor și, în final, ieșirea rețelei;
- o etapă de propagare înapoi (*backward pass*), în care eroarea calculată la nivelul stratului de ieșire este propagată în sens invers, dinspre ieșire spre intrare, atribuind fiecărui neuron o cotă de responsabilitate pentru eroarea totală și permițând astfel calcularea gradientului pentru fiecare pondere.

Cele două etape se repetă pentru exemplele setului de antrenament până la convergența erorii către un minim. Deoarece suprafața de eroare nu este convexă, algoritmul nu garantează atingerea minimului global, putând rămâne blocat într-un minim local; în practică, însă, soluțiile obținute oferă o capacitate de generalizare satisfăcătoare.

Eficiența și stabilitatea procesului depind în mod esențial de alegerea ratei de învățare și de strategia de actualizare a ponderilor, care poate fi aplicată după fiecare exemplu, pe loturi reduse de exemple (*mini-batch*) sau asupra întregului set de antrenament.

Paradigme de utilizare în recunoașterea caracterelor

Figura 4.6 prezintă principalele două paradigme de utilizare a rețelelor neuronale în cadrul sistemelor de recunoaștere a caracterelor. Prima dintre ele implică un extractor de trăsături explicit, care transmite mai departe rezultatele sale la stratul de intrare al rețelei neuronale. Paradigma alternativă nu extrage în mod explicit trăsăturile caracterelor, acest lucru se întâmplă în mod implicit în cadrul straturilor ascunse. O proprietate avantajoasă a acestei paradigme este faptul că extragerea trăsăturilor și clasificarea sunt antrenate în mod simultan pentru a produce rezultate optime.

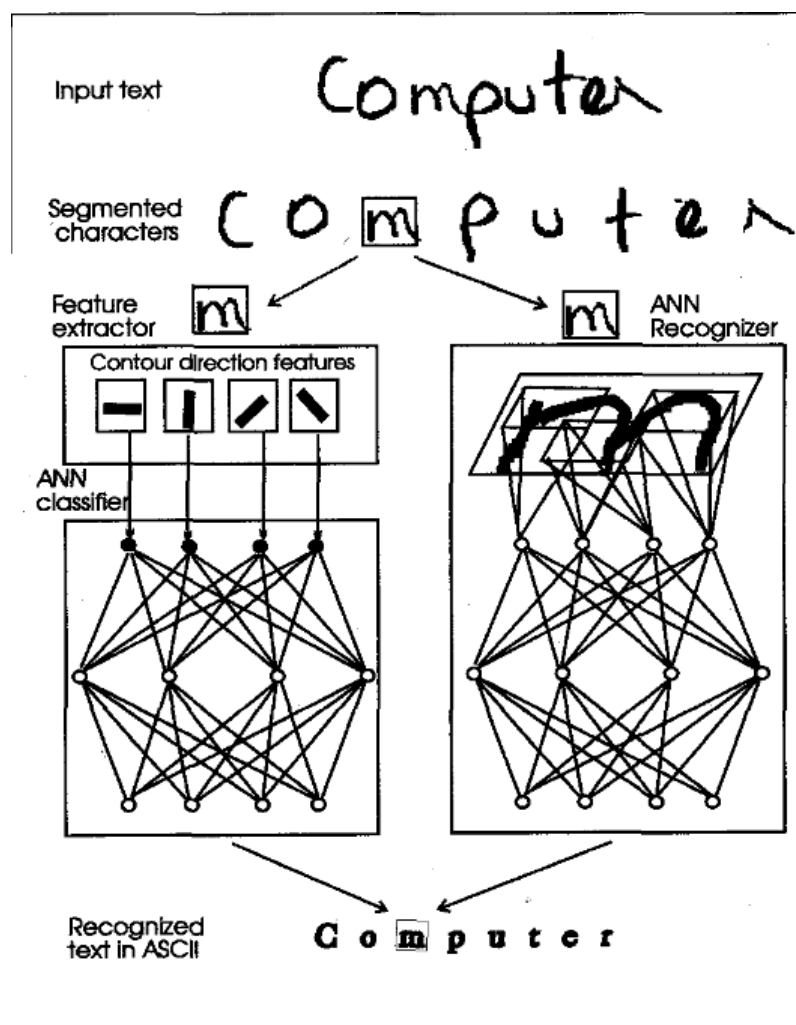


Figura 4.6: Două variante pentru folosirea ANN-urilor în contextul OCR-ului [24]

Capitolul 5

Implementarea sistemului IORA

Sistemul propus în lucrarea de față, denumit IORA, a fost construit pornind de la o arhitectură modulară formată din trei componente cheie: modulul de preluare a datelor (*Input Module*), fluxul de procesare (*Processing Pipeline*) și modulul de logică a aplicației (*Application Logic Module*). Pentru a oferi flexibilitate și a respecta principiile de proiectare extensibilă specifice conceptului de *clean code* [27], modulul de intrare și cel de logică a aplicației utilizează polimorfismul prin expunerea unor interfețe abstracte.

Această abordare decuplată permite diferitelor aplicații să integreze nucleul sistemului într-un mod elegant, implementând comportamente personalizate și adaptate la mediul de execuție, fără a modifica codul de bază. De exemplu, modulul de intrare poate fi extins pentru a prelua fluxuri video de la o cameră atașată mediului de execuție, fie prin intermediul *socket-urilor*. În mod similar, modulul de logică a aplicației poate fi personalizat pentru a declanșa acțiuni specifice precum trimiterea unor notificări automate prin *e-mail*, sau acționarea unor echipamente *hardware*, în cazul unui sistem integrat.

În contrast, fluxul de procesare este încapsulat ca o unitate de execuție independentă și de sine stătătoare, neexpunând nicio interfață publică destinată extinderii structurale.

În continuare, acest capitol descrie arhitectura și tehnologiile utilizate în implementarea aplicației prezentate alături de o descriere tehnică a fiecărui modul.

5.1 Arhitectura sistemului

Sistemul IORA este organizat pe trei niveluri logice care corespund celor trei module introduse la începutul capitolului: nivelul de achiziție a datelor, nivelul de procesare și nivelul de logică a aplicației. Această stratificare urmărește o separare clară a responsabilităților, fiecare nivel comunicând cu cel adiacent exclusiv prin

structuri de date simple, independente de bibliotecile externe folosite în implementare. Decuplarea astfel obținută permite ca un nivel să fie înlocuit sau extins fără a afecta restul sistemului.

5.1.1 Diagrama de componente

Diagrama de componente, prezentată în Figura 5.1, surprinde cele trei module și fluxul de date dintre ele. Modulul de intrare expune interfața `ICaptureManager` și produce, indiferent de sursa fizică a datelor, obiecte de tip `Frame`. Aceste obiecte constituie singurul punct de contact dintre modulul de intrare și fluxul de procesare, eliminând orice dependență a nucleului de natura sursei de captură.

Fluxul de procesare consumă obiectul `Frame` și aplică, în mod secvențial, cele trei etape ale procesului de recunoaștere: localizarea plăcuței, segmentarea caracterelor și recunoașterea acestora. Rezultatul produs (șirul de caractere recunoscut, însoțit de metadatele asociate) este transmis mai departe către modulul de logică a aplicației, care decide acțiunea declanșată în funcție de contextul de execuție.

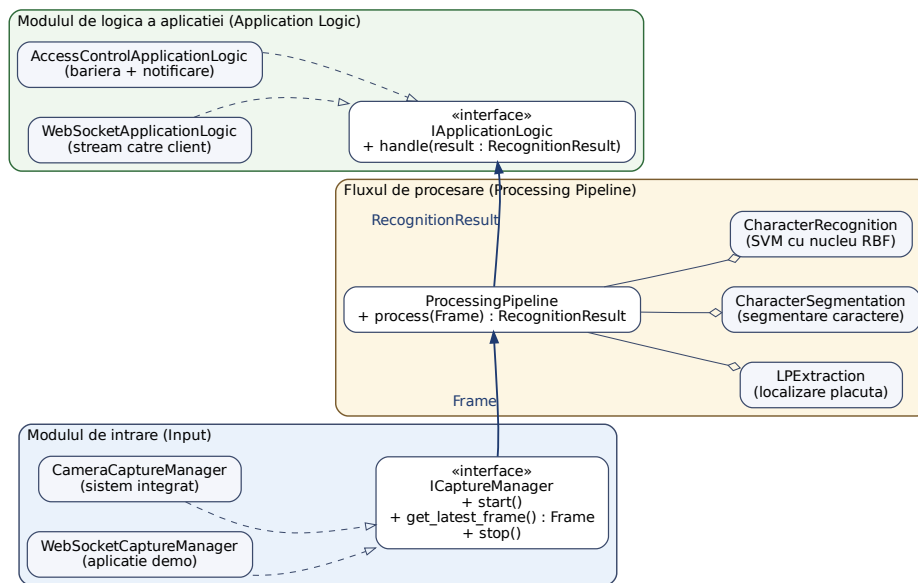


Figura 5.1: Diagrama de componente a sistemului IORA: modulele de intrare, procesare și logică a aplicației, împreună cu structurile de date `Frame` și `RecognitionResult` care le decuplează.

5.1.2 Actori și cazuri de utilizare

În cadrul scenariului de utilizare vizat, sistemul interacționează cu mai mulți actori, atât umani, cât și automați. Actorul uman principal este operatorul, responsa-

bil de configurarea și monitorizarea sistemului. Actorii automați sunt reprezentați de echipamentele și serviciile care consumă rezultatul recunoașterii: sistemul de barieră (acționat în urma unei recunoașteri valide) și serviciul de notificare (care transmite alerte, de exemplu prin *e-mail*).

Principalele cazuri de utilizare sunt: monitorizarea fluxului video de la sursa de captură, citirea automată a numărului de înmatriculare dintr-un cadru și declanșarea unei acțiuni asociate (ridicarea barierei sau emiterea unei notificări) pe baza rezultatului recunoașterii. Pentru aplicația demonstrativă, se adaugă cazul de utilizare al transmiterii rezultatelor către un client extern prin intermediul unei conexiuni *Web-Socket*. Aceste cazuri de utilizare, împreună cu actorii implicați, sunt sintetizate în Figura 5.2.

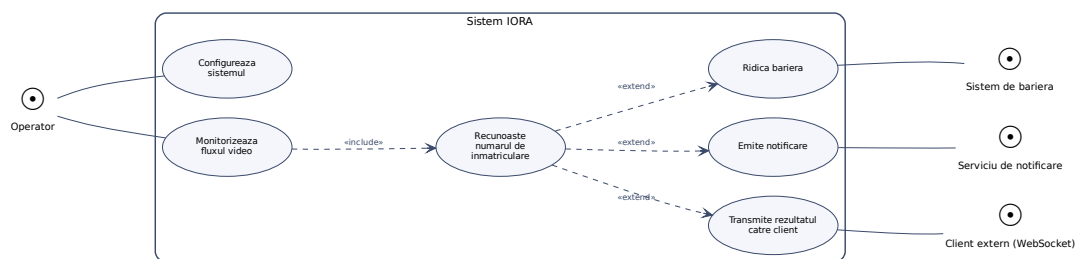


Figura 5.2: Diagrama cazurilor de utilizare a sistemului IORA, cu operatorul drept actor uman principal și sistemele consumatoare ale rezultatului recunoașterii drept actori automați.

5.1.3 Diagrama de clase

Diagrama de clase din figura 5.3 reflectă, la nivel structural, aceeași stratificare introdusă de diagrama de componente: interfețele `ICaptureManager` și `IApplicationLogic` delimitează marginile extensibile ale sistemului, fiecare având două realizări concrete (una pentru sistemul integrat, una pentru aplicația demonstrativă), în timp ce nucleul de procesare este modelat prin clasa `ProcessingPipeline`, care agregă cele trei etape ale recunoașterii (`LPExtraction`, `CharacterSegmentation`, `CharacterRecognition`). Tipurile de date proprii `Frame` și `RecognitionResult` apar ca dependențe la marginile fluxului, marcând explicit decuplarea nucleului de detaliile de implementare ale surselor de captură și ale acțiunilor declanșate de aplicație.

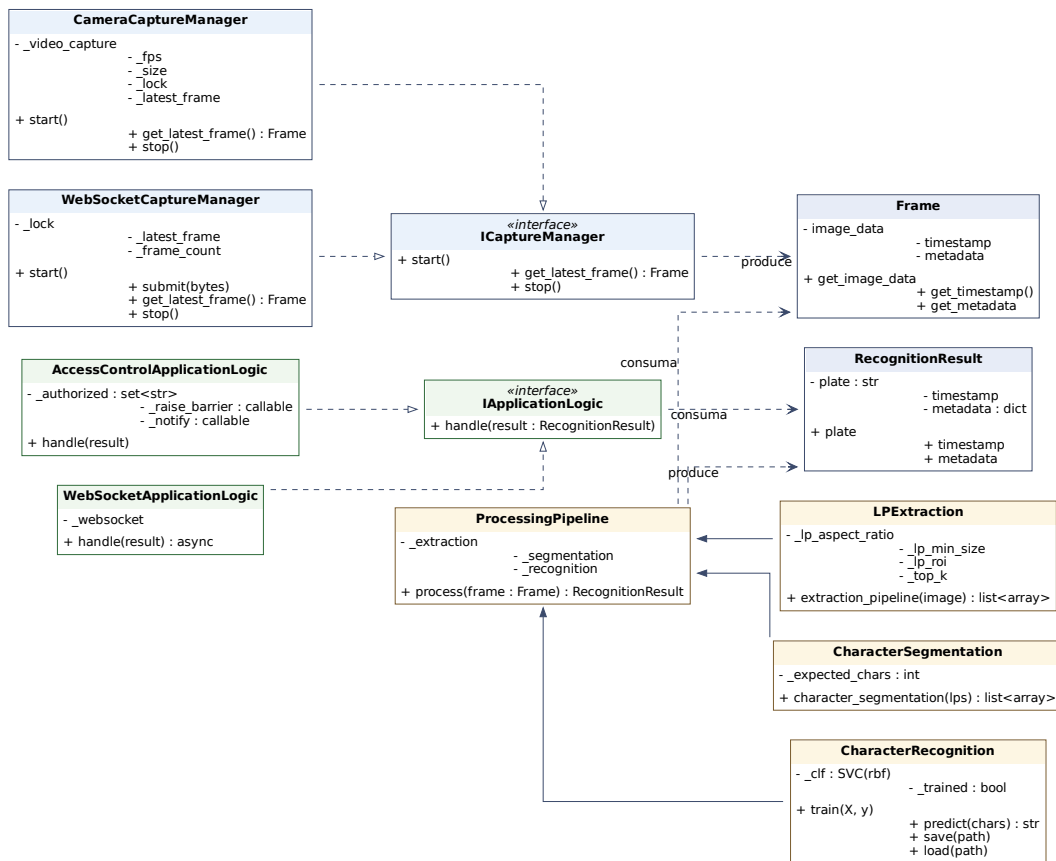


Figura 5.3: Diagrama de clase a sistemului IORA: interfețele extensibile, realizările lor concrete și agregarea celor trei etape de procesare în **ProcessingPipeline**.

5.1.4 Decizii de proiectare

Proiectarea bazată pe principiile arhitecturale menționate anterior sugerează natura componentelor: modulele de la margini interacționează direct cu mediul de execuție și sunt, prin urmare, cele mai susceptibile la variații, motiv pentru care comportamentul lor este abstractizat prin interfețe conform principiilor de *clean code* [27]. În schimb, fluxul de procesare implementează un algoritm stabil, independent de context, care nu beneficiază de pe urma extensibilității.

O a doua decizie de proiectare vizează minimizarea dependențelor la nivelul nucleului și definirea unor tipuri de date proprii. Această alegere consolidează portabilitatea sistemului și permite adoptarea sa în contexte de rulare diferite, de la sisteme integrate cu resurse limitate până la servere care deservește o aplicație web.

5.2 Tehnologii

Alegerea limbajului a fost făcută pe considerente practice, așadar întreg sistemul a fost implementat în Python 3.14.4 în detrimentul unui limbaj compilat precum C++. Această decizie a fost motivată de gradul ridicat de adopție a ecosistemului Python în cadrul sistemelor integrate, precum și de flexibilitatea și viteza de dezvoltare specifice oferite de un limbaj dinamic. De asemenea, librăriile necesare implementării modului de procesare se bazează pe nuclee scrise în C.

Pentru a oferi o structură clară, tehnologiile utilizate au fost împărțite în două categorii: cele fundamentale, necesare dezvoltării nucleului și cele adiționale, specifice dezvoltării aplicației finale.

5.2.1 Nucleu

Din punct de vedere tehnologic, implementarea nucleului se bazează pe un ecosistem minimalist de dependențe. Tehnologiile de bază utilizate sunt:

- OpenCV 4.13.0
- numpy 2.4.2
- scikit-learn 1.8.0
- matplotlib-base 3.10.9

5.2.2 Nivelul de aplicație

După cum am menționat, dependențele necesare dezvoltării aplicației finale sunt opționale, acestea depinzând strict de contextul aplicației dorite. Pentru aplicația propusă, dependențele sunt:

- fastapi 0.135.4
- websockets 15.0.1

5.3 Detalii tehnice

În cele ce urmează va fi prezentată o descriere detaliată a arhitecturii fiecărui modul, abordând aspecte precum interfețele de programare expuse, deciziile de proiectare și implementarea propriu-zisă.

Pentru modulele extensibile, se va realiza o paralelă între implementarea în cadrul unui sistem integrat și implementarea concretă realizată în cadrul aplicației demonstrate.

5.3.1 Modulul de intrare

Achiziția datelor de intrare reprezintă punctul de plecare al sistemului. Acest modul are rolul exclusiv de a prelua informația brută și de a o expune către fluxul de procesare. În continuare vor fi prezentate elementele atât abstracte cât și concrete ale acestei componente.

Interfața `ICaptureManager`

Interfața `ICaptureManager` este definită prin trei metode esențiale. Primele două, `start` și `stop`, gestionează ciclul de viață al sistemului de captură, ocupându-se de inițializarea și, respectiv, de eliberarea resurselor asociate acestuia. Cea de-a treia metodă, `get_latest_frame`, returnează cel mai recent cadru preluat, încapsulat într-un obiect de tip `Frame`.

```
def start(self) -> None:
    pass

def get_latest_frame(self) -> Frame:
    pass

def stop(self) -> None:
    pass
```

Sistem integrat

Pentru scenariul rulării pe un sistem integrat, a fost realizată o implementare exemplificativă, clasa `CameraCaptureManager`, responsabilă de preluarea cadrelor de la o cameră conectată direct la dispozitiv.

Implementarea se bazează pe biblioteca `OpenCV` pentru a media interacțiunea cu subsistemul de captură video al sistemului de operare. Identificarea sursei fizice se realizează prin intermediul indexului de dispozitiv asociat camerei, transmis ca parametru constructorului clasei.

Pe lângă indexul de dispozitiv, clasa expune următoarele atribute private:

- `_video_capture` - obiectul `OpenCV` prin care se realizează comunicarea cu camera;
- `_fps` - numărul de cadre pe secundă raportat de dispozitiv;
- `_size` - rezoluția cadrelor preluate, dedusă din primele cadre citite;
- `_lock` - obiect necesar pentru sincronizarea accesului la cel mai recent cadru.

Metoda `get_latest_frame` a fost proiectat astfel încât să returneze întotdeauna cel mai recent cadru disponibil. Această cerință este esențială într-un context de procesare în timp real: întrucât bibliotecile de captură mențin o memorie internă, citirea secvențială a cadrelor ar introduce o latență cumulativă, returnând cadre deja învechite. Pentru a evita acest fenomen, un fir de execuție (*thread*) dedicat preia continuu cadrele de la cameră, păstrând disponibil doar cel mai recent dintre acestea. Accesul concurent la resursa partajată este sincronizat printr-un mecanism de tip *lock*.

Aplicație demonstrativă

În cadrul aplicației demonstrative, sursa de captură nu este o cameră atașată direct dispozitivului, ci un flux video transmis în timp real (*live-feed*) printr-o conexiune *WebSocket*. Această implementare menține o conexiune persistentă prin care cadrele sosesc continuu de la un client extern, le decodează și le încapsulează în același obiect `Frame`. La fel ca în cazul implementării pentru sistemul integrat, doar cel mai recent cadru recepționat este păstrat disponibil, evitând acumularea unei latențe în contextul procesării în timp real.

5.3.2 Fluxul de procesare

Considerații preliminare

Precum am menționat când am definit contextul aplicației, există o diversitate ridicată în privința formatului, a paletelor cromatice de fond, a dimensiunilor relative și a fontului caracterelor. Tocmai din aceste motive, în prezenta lucrare a fost exclusă de la bun început orice abordare bazată pe segmentarea culorii, deși aceasta este frecvent utilizată în literatura de specialitate. Generalizarea acestor abordări către scenariul nostru ar fi necesitat menținerea unui catalog de palete pentru fiecare format admis, ceea ce ar fi introdus o sursă suplimentară de erori și ar fi diminuat capacitatea de adaptare a sistemului.

Localizarea plăcuței de înmatriculare

În capitolul teoretic dedicat acestui sub-sistem au fost menționate trei abordări: bazate pe culoare, pe contururi sau pe textură. După cum am menționat anterior, cele bazate pe culoare nu sunt o variantă fiabilă deoarece pe teritoriul României există patru combinații cromatice posibile care definesc un număr de înmatriculare valid.

Pe de altă parte, metodele bazate pe textură prezintă un grad al robusteții mai scăzut față de cele bazate pe contururi. Deși ambele categorii pornesc de la aceeași observație, anume că zona caracterelor produce o concentrare de tranziții de intensitate, ele diferă fundamental prin criteriul de decizie folosit pentru a confirma o regiune drept plăcuță.

Abordările texturale se folosesc de prezența caracterelor drept caracteristică principală a numărului de înmatriculare. Acest descriptor nu este unul robust pentru cazul general, deoarece văile care separă două caractere diferă de la format la format. Totodată, sensibilitatea la zgomot reprezintă o limitare majoră a acestor metode. O imagine degradată sau prezența unor umbre poate compromite analiza proiecțiilor verticale, ducând la detectarea eronată a unor văi inexistente.

În contrast, metodele bazate pe contururi tratează concentrarea de tranziții doar ca pe un indiciu, lăsând decizia finală în seama unei constante geometrice: forma dreptunghiulară cu un raport de aspect cunoscut. Deoarece acest criteriu nu depinde de modul în care sunt distribuite caracterele, ci de forma regiunii, el rămâne stabil indiferent de fontul, dimensiunea sau numărul caracterelor.

Pornind de la aceste considerente, etapa de localizare a fost implementată în clasa `LPExtraction`, parametrizată prin raportul de aspect admis pentru o plăcuță (`lp_aspect_ratio`), aria minimă a unei regiuni candidat (`lp_min_size`), o regiune de interes opțională (`lp_roi`) și numărul de candidați returnați (`top_k`). Fluxul de procesare urmează etapele clasice ale abordării pe contururi, descrise în capitolul 2, adaptate contextului autohton.

Preprocesarea începe, dacă este definită, cu decuparea regiunii de interes, urmată de conversia în tonuri de gri și de aplicarea unui filtru median pentru atenuarea zgomotului. Asupra imaginii filtrate se aplică operatorul Sobel pe direcția orizontală, evidențiind muchiile verticale generate de caractere.

Rezultatul este binarizat prin metoda Otsu în variantă inversată conformându-se astfel cu formatul așteptat de funcția de extragere a contururilor. Asupra imaginii binare se aplică o operație morfologică de deschidere, urmată de o dilatare cu un nucleu orizontal a cărui lățime este direct proporțională cu lățimea imaginii. Scopul dilatării este de a uni elementele plăcuței care au fost separate în pașii anteriori.

După toate acestea se extrag contururile externe, iar dreptunghiurile lor de încadrare sunt filtrare pe baza raportului de aspect și a ariei minime, eliminând regiunile necorespunzătoare.

```
greyscale = cv.cvtColor(array, cv.COLOR_BGR2GRAY)
median = self._helper.median_filter(array=greyscale)
sobel = self.sobel(median)
thresh = self._helper.otsu_thresholding(sobel)
thresh = cv.bitwise_not(thresh)
```



```

open_image = self._helper.opening(thresh)
dilated_image = self._helper.dilation(open_image)
bounding_box = self._helper.find_countours(
    dilated_image,
    self._lp_aspect_ratio,
    self._lp_min_size
)

```

Datorită experimentelor a putut fi observat faptul că, în majoritatea cazurilor plăcuțele erau fragmentate în două sau chiar trei bucăți. Așadar a fost necesară implementarea unei metode de reuniune a acestor fragmente pornind de la poziția lor în cadrul imaginii. Fiecare dreptunghi de încadrare extras de pasul anterior este extins în interiorul imaginii cu o margine fixă, iar dacă două dintre dreptunghiurile inițiale se suprapun în urma extinderii ele sunt unite.

În final, candidații rămași sunt ordonați după densitatea de muchii din interiorul lor, iar primii `top_k` sunt returnați. Returnarea mai multor candidați, în locul unuia singur, constituie o decizie de proiectare deliberată: ea permite etapei următoare de segmentare să recupereze eventualele erori de localizare, alegând candidatul care conduce la cea mai plauzibilă segmentare.

Rezultatul acestei etape poate fi observat în Figura 5.4 care prezintă cele trei regiuni de interes extrase de modul în momentul în care acesta este construit cu parametrul `top_k=3`.



Figura 5.4: Imaginea inițială și cele 3 elemente extrase în momentul rulării modului de localizare cu `top_k=3`.

Segmentarea caracterelor

Etapa de segmentare primește candidații de plăcuță produși de etapa de localizare și are ca scop izolarea fiecărui caracter într-o imagine distinctă. Dintre tehnicile prezentate în capitolul 3, a fost adoptată analiza contururilor, combinată cu utilizarea cunoștințelor apriori privind aspectul caracterelor. Această alegere a fost preferată unei abordări bazate pe analiza proiecțiilor care, deși oferă o reprezentare unidimensională compactă a distribuției pixelilor de prim-plan, depinde de

existența unor văi suficient de adânci între caracterele adiacente. Pe plăcuțele reale, înclinarea de câteva grade și spațierea neuniformă a caracterelor fac ca aceste văi să se estompeze, fuzionând simboluri vecine într-un singur segment. Decizia la nivelul fiecărei componente conexe în parte rămâne, în schimb, independentă de unghiul de înclinare și de spațierea dintre simboluri, comportament confirmat de comparația cantitativă prezentată în secțiunea de experimente.

Pipeline-ul de segmentare începe cu restrângerea decupajului la corpul efectiv al plăcuței, eliminând contextul reținut eronat în etapa de localizare, precum fragmentele de caroserie sau cadrul plăcuței. Restrângerea se realizează fie prin selectarea celei mai mari componente conexe luminoase, corespunzătoare fundalului uniform al plăcuței, fie, atunci când aceasta nu este concludentă, prin detectarea benzii de caractere: elementele de înălțime plauzibilă sunt grupate după coordonata lor verticală, iar decupajul este redus la dreptunghiul de încadrare al grupului dominant.

Corpul astfel obținut este binarizat printr-un prag adaptiv gaussian în variantă inversată (`adaptiveThreshold`), care compensează variațiile locale de iluminare de-a lungul plăcuței, obținând caracterele drept prim-plan alb. Din imaginea binară se extrag contururile externe, iar dreptunghiurile lor de încadrare constituie casetele-candidat pentru caractere. Atunci când numărul de contururi este insuficient (sub trei), imaginea este erodată ușor, iar extragerea este reluată, separând caracterele unite accidental prin zgomot.

Casetele-candidat sunt apoi filtrate printr-o succesiune de criterii derivate din aspectul așteptat al unui caracter. Un prag de contrast intern, calculat ca abaterea standard a intensităților din interiorul casetei raportată la un prag Otsu determinat pe ansamblul casetelor, elimină regiunile uniforme precum reziduurile cadrului. Constrângerile de înălțime relativă (între 0,40 și 0,95 din înălțimea plăcuței) și de raport de aspect (w/h între 0,08 și 1,3) elimină structurile prea mici sau cu geometrie implauzibilă, pragul inferior fiind suficient de permisiv pentru a păstra caractere înguste precum cifra 1. În final, casetele care ating marginile laterale ale decupajului sunt respinse, anulând fragmentele de cadru tăiate, iar o trecere de consistență păstrează doar casetele a căror înălțime se află în jurul valorii mediane (între 0,55 și 1,45 din aceasta).

```
thresholded = cv.adaptiveThreshold(  
    greyscale, 255, cv.ADAPTIVE_THRESH_GAUSSIAN_C,  
    cv.THRESH_BINARY_INV, block, 2)  
contours = self._helper.find_countours(  
    thresholded, aspect_ratio_bounds=(0.04, 2.0))  
for c, s in zip(contours, stds):  
    x, y, w, h = c  
    if s < std_thr:
```

```
        continue
    if not (min_height <= h <= max_height):
        continue
    if not (min_w_h <= w / h <= max_w_h):
        continue
    if x < border_margin or (x + w) > (W - border_margin):
        continue
    kept.append((x, y, w, h))
```

Caracterele lipite, care produc o singură casetă vizibil mai lată decât înaltă ($w > 1,2h$), sunt separate ulterior prin metoda `_split_wide_blobs`: în interiorul casetei se calculează proiecția verticală pe coloane, iar tăierea se face în dreptul minimelelor locale (văile dintre caractere), numărul de tăieturi fiind estimat din raportul de aspect al casetei. Spre deosebire de o abordare globală bazată pe proiecții, profilul este aici folosit strict local, pentru a împărți o regiune deja izolată, evitând astfel sensibilitatea profilului global la înclinarea plăcuței.

Deoarece etapa de localizare poate furniza mai mulți candidați de plăcuță, segmentarea este rulată pe fiecare dintre aceștia, iar setul de caractere reținut este cel care obține scorul maxim conform unei funcții de evaluare (`_score`). Acest cuplaj prin parametrul `top_k` transformă etapa de segmentare într-un mecanism de recuperare a erorilor de localizare: un candidat incorect (de exemplu o regiune de grilaj reținută eronat) va produce o segmentare cu un scor scăzut și va fi astfel respins în favoarea candidatului corect.

Funcția de evaluare valorifică o cunoștință apriori esențială: o plăcuță validă conține un număr cunoscut de caractere, de dimensiuni comparabile și dispuse pe aceeași linie de bază. Scorul integrează, în consecință, trei componente: abaterea numărului de casete detectate față de numărul așteptat (în jur de șapte simboluri pentru formatul autohton), consistența înălțimilor casetelor (caracterele unei plăcuțe au înălțimi similare) și alinierea lor verticală (caracterele împărtășesc aceeași linie de bază). O segmentare care satisface aceste constrângeri primește un scor ridicat, întrucât corespunde tiparului geometric al unei plăcuțe reale.

Rezultatul acestei etape este ilustrat în Figura 5.5: pornind de la plăcuța localizată în etapa anterioară, modulul izolează fiecare caracter într-o imagine distinctă, producând cele șapte simboluri care urmează a fi transmise individual etapei de recunoaștere.



(a) Plăcuța localizată



Figura 5.5: Rezultatul etapei de segmentare: plăcuța localizată (sus) și cele șapte caractere izolate (jos), fiecare extras într-o imagine distinctă.

Recunoașterea caracterelor

Ultima etapă a fluxului de procesare atribuie fiecărui caracter segmentat o etichetă, dintr-un alfabet de cifre și litere. Pentru această sarcină a fost ales un clasificator de tip SVM, ale cărui fundamente teoretice au fost prezentate în capitolul 4.

Fiecare caracter este adus la o reprezentare uniformă, independentă de dimensiunea sa originală. Imaginea este mai întâi adusă la o formă pătrată prin completarea laturii mai scurte cu o bordură, apoi redimensionată la o rezoluție fixă de 28×28 pixeli. Asupra acestei imagini se calculează un descriptor de tip HOG, pe o fereastră de 28×28 cu celule de 7×7 pixeli, blocuri de 14×14 cu pas de 7 pixeli și 9 intervale de orientare, rezultă un vector de 324 de trăsături. Această reprezentare, care surprinde distribuția locală a orientărilor muchiilor în loc de intensitatea brută a pixelilor, constituie intrarea clasificatorului.

Predictia nu se reduce la atribuirea independentă a etichetei celei mai probabile fiecărui caracter, ci valorifică o cunoștință apriori privind formatul plăcuțelor. Atunci când numărul de caractere segmentate corespunde unui format cunoscut, decodarea se realizează poziție cu poziție, restrângând la fiecare poziție mulțimea etichetelor admise (cifră sau literă) și alegând eticheta cu marginea maximă raportată de funcția de decizie a clasificatorului. Pentru lungimile ambigue, în care același număr de caractere admite mai multe tipare, fiecare tipar este evaluat separat, iar varianta cu marginea medie cea mai mare este reținută. Suplimentar, atunci când segmentarea produce un caracter în plus față de un format așteptat, se încearcă și subsecvențele obținute prin eliminarea pe rând a câte unui caracter, comparate tot pe baza marginii medii per poziție; astfel, un fragment supra-segmentat, care primește o margine apropiată de zero, este eliminat în favoarea tiparului care se potrivește curat. Dacă numărul de caractere nu corespunde niciunui format cunoscut, se revine la atribuirea neconstrânsă a etichetei celei mai probabile.

5.3.3 Nivelul de aplicație

Modulul de logică a aplicației constituie, alături de modulul de intrare, una dintre marginile extensibile ale sistemului. Rolul său este de a consuma rezultatul produs de fluxul de procesare și de a declanșa acțiunea corespunzătoare contextului de execuție.

Sistem integrat

În scenariul unui sistem integrat, modulul de logică a aplicației acționează direct asupra mediului fizic. O implementare reprezentativă este cea a unui punct de control al accesului, în care recunoașterea unui număr de înmatriculare aflat într-o listă de autorizare declanșează ridicarea unei bariere auto prin acționarea echipamentului *hardware* corespunzător. Aceeași implementare poate emite, în paralel, notificări automate, de exemplu prin *e-mail*, către un operator sau către un sistem de evidență, atât pentru accesele autorizate, cât și pentru tentativele neautorizate. Caracterul abstract al interfeței permite combinarea liberă a unor astfel de acțiuni, fără modificarea nucleului.

Aplicație demo

Pentru aplicația demonstrativă, modulul de logică a aplicației nu acționează asupra unui echipament fizic, ci expune rezultatele recunoașterii către un client extern. Implementarea se bazează pe biblioteca *FastAPI*, care oferă infrastructura unui server web, și pe protocolul *WebSocket*, ales pentru capacitatea sa de a menține o conexiune bidirecțională persistentă. Prin intermediul acestei conexiuni, rezultatele procesării sunt transmise în timp real către interfața clientului, pe măsură ce sunt produse de fluxul de procesare, fără a necesita interogări repetate din partea acestuia.

Capitolul 6

Metrici și experimente

Capitolul de față prezintă evaluarea experimentală a sistemului IORA, urmărind să cuantifice performanța fiecărei etape a fluxului de procesare, dar și a întregului lanț de recunoaștere. Mai întâi sunt descrise seturile de date utilizate, atât cel destinat evaluării *end-to-end* a sistemului, cât și cel folosit pentru antrenarea clasificatorului de caractere, împreună cu procedeele de normalizare, adnotare și augmentare aplicate acestora. În continuare sunt introduse metricile de evaluare și sunt raportate rezultatele obținute, mai întâi pentru componenta de recunoaștere a caracterelor considerată izolat, apoi pentru sistem la nivel de etapă și *end-to-end*. În final, secțiunea de experimente justifică, prin rezultate cantitative, principalele decizii de proiectare adoptate în fluxul de procesare și evidențiază contribuția fiecărei intervenții la performanța de ansamblu.

6.1 Seturi de date

Sistemul IORA operează cu două seturi de date distincte: un set de evaluare, utilizat pentru măsurarea performanței *end-to-end* a fluxului de procesare, și un set de antrenare, destinat construirii clasificatorului SVM. Ambele gravitează în jurul aceleiași corpus de imagini cu plăcuțe de înmatriculare obținut prin platforma Roboflow [28], completat, în cazul setului de evaluare, de o componentă românească colectată manual, iar în cazul setului de antrenare, de un set de date dedicat caracterelor individuale cu fonturi variate.

6.1.1 Setul de evaluare

Pentru a determina robustețea sistemului și capacitatea sa de adaptare la diverse formate ale numărului de înmatriculare a fost necesară crearea unui set de date mixt, în care sunt prezente atât autovehicule de pe teritoriul României, cât și vehicule din alte regiuni ale globului. Pentru prima componentă a setului de date în momentul

de față nu există o sursă general valabilă și deschisă publicului larg, așadar imaginile reprezentative acestei părți au fost colectate manual de pe platforme de comerț auto online. Această procedură de salvare și utilizare a imaginilor este conformă cu reglementările GDPR, întrucât fotografiile au fost publicate în mod voluntar de către vânzători pe platforme cu acces liber, fiind astfel destinate publicului larg, iar prelucrarea lor s-a realizat strict în scop de cercetare și evaluare a sistemului. Pentru componenta internațională, a fost utilizat setul de date public obținut prin intermediul platformei Roboflow [28].

Normalizarea seturilor de date

În contextul sistemelor de recunoaștere automată a numerelor de înmatriculare puse în funcțiune în cadrul parcărilor condițiile de execuție precum poziția numărului de înmatriculare, distanța și raportul dimensiunilor, unghiul de captare și distanța focală sunt bine cunoscute, permitând valorificarea lor în procesul de localizare.

Pentru a reproduce aceste condiții, a fost necesară normalizarea setului de date. În cazul imaginilor obținute în mod manual, acest procedeu a presupus selectarea autovehiculelor care prezentau un număr de înmatriculare vizibil, urmată de o decupare pentru a reduce zgomotul de fundal și a oferi o distanță focală relativ constantă. Un exemplu al procesului poate fi observat în figura 6.1, unde înainte de normalizare erau prezente elemente de zgomot agresive atât în parbrizul mașinii cât și în spațiul înconjurător.



(a) Înainte de normalizare



(b) După normalizare

Figura 6.1: Un exemplu de imagine alterată pentru a reduce zgomotul de fundal și normaliza unghiul de captură

Pentru componenta internațională a setului de date procesul a fost unul mai rapid, fiind necesară doar selectarea imaginilor corespunzătoare criteriilor menționate anterior. În figura 6.2 se poate observa un exemplu de imagine corespunzătoare pusă în paralel cu o imagine care nu adera la standardele impuse.



(a) Imagine corespunzătoare



(b) Imagine necorespunzătoare

Figura 6.2: Paralelă între o imagine conformă și una neconformă

Crearea sursei de adevăr a setului de date

Procedeul descris anterior constituie, însă, doar primul pas în construirea unui set de date potrivit strategiei de procesare propuse de această teză. Este în continuare necesară o sursă de adevăr uniformă pe componentele locală și internațională. Această secțiune prezintă pașii luați în completarea imaginilor cu informații relevante și conforme cu realitatea, care să reprezinte reperul față de care vor fi evaluate predicțiile generate de *pipeline*.

Prima etapă este reprezentată de adnotarea imaginilor cu numărul de înmatriculare afișat în acestea, sarcină realizată manual, prin deschiderea lor rând pe rând, urmată de adăugarea valorilor într-un fișier `.csv` dedicat. Pentru a permite analizarea performanței componentei de localizare, a fost dezvoltat un utilitar *software* dedicat extragerii chenarelor ce încadrează numerele de înmatriculare (*bounding boxes*). Această aplicație cu interfață grafică facilitează delimitarea manuală a regiunii de interes, permițând utilizatorului să definească chenarul de încadrare prin marcarea coordonatelor aferente colțurilor din stânga-sus și dreapta-jos ale plăcuței.

6.1.2 Setul de antrenare a clasificatorului SVM

Antrenarea clasificatorului de caractere necesită un set de date format din imagini de caractere individuale, fiecare asociată cu eticheta sa. Construirea acestui set a urmat o strategie pe două surse: caractere generate automat prin aplicarea fluxului de procesare asupra imaginilor de antrenare din setul [28] de plăcuțe, urmate de etichetare manuală prin intermediul unei aplicații *software* dedicate, și un set extern de caractere individuale extrase din imagini naturale, cu o mare diversitate de fonturi, scale și condiții de captură [29]. Combinarea celor două surse urmărește, pe de o parte, alinierea distribuției datelor de antrenare la caracteristicile particulare ale caracterelor segmentate de sistemul propriu și, pe de altă parte, asigurarea unui volum și a unei diversități tipografice suficiente.

Generarea automată prin fluxul de procesare

Prima sursă de date este generată prin intermediul sistemului cu ajutorul utilitarului *software* `generate_dataset.py`

Acesta aplică asupra imaginilor de antrenare din componenta internațională a setului de plăcuțe etapele de localizare și segmentare descrise anterior, salvând caracterele segmentate drept imagini individuale, neetichetate. Integrarea acestui subset autogenerat în datele de antrenament a fost motivată de considerente practice: exemplele astfel obținute reflectă cu maximă fidelitate particularitățile și imperfecțiunile specifice extracției reale din sistem. Astfel, clasificatorul este expus la condiții realiste de operare, învățând să gestioneze variații ale unghiului de rotație, artefacte de segmentare sau caractere trunchiate.

Sursă externă de caractere

A doua sursă de date o constituie setul public *Chars74K* [29], introdus de de Campos și colaboratorii pentru problema recunoașterii caracterelor în imagini naturale. Dintre componentele acestui set a fost utilizat subsetul *EnglishImg*, care reunește 7705 caractere segmentate din scene naturale, distribuite în 62 de clase corespunzătoare cifrelor și literelor latine majuscule și minuscule (0–9, A–Z, a–z). Spre deosebire de caracterele produse de propriul *pipeline*, aceste mostre provin din imagini reale, prezentând o variabilitate ridicată a fontului, a scalei și a condițiilor de iluminare, întrucât autorii au păstrat rezoluția originală a fiecărui caracter așa cum apare în imaginea-sursă.

Setul este distribuit deja etichetat și organizat ca un arbore de directoare, cu câte un director per clasă, fiecare caracter fiind salvat ca o imagine PNG individuală. Această structură permite integrarea directă a mostrelor în setul de antrenare, fără efort manual de adnotare. Rolul acestei surse este de a completa diversitatea tipografică a caracterelor autogenerate: în timp ce prima sursă aliniază distribuția datelor la particularitățile segmenterului propriu, *Chars74K* introduce o varietate mai amplă de forme și de fonturi întâlnite în condiții reale, contribuind la capacitatea de generalizare a clasificatorului.

Etichetarea manuală

Etichetarea mostrelor generate automat este realizată prin scriptul `label_dataset.py`, care oferă o interfață minimală de etichetare asistată, construită cu ajutorul bibliotecii OpenCV. Fiecare imagine neetichetată este afișată succesiv, iar operatorul atribuie eticheta prin apăsarea tastei corespunzătoare caracterului. Apoi, imaginea este mutată automat în directorul clasei respective. Tasta dedicată permite marcarea mos-

trelor nevalide (caractere nerecunoscute sau segmentate eronat) prin mutarea lor într-un director special, în timp ce o altă tastă permite ignorarea unei mostre.

Augmentarea setului prin rotații

Deoarece setul de caractere etichetate manual prezenta un volum limitat și o variație redusă a înclinației, a fost necesară extinderea artificială a acestuia. Această etapă s-a realizat prin intermediul utilitarului software `augment_dataset`, care aplică fiecărei imagini o rotație în jurul centrului imaginii, la unghiurile de $\pm 3^\circ$ și $\pm 5^\circ$. Acest proces nu are doar scopul de a extinde mărimea setului de date, ci și de a forța clasificatorul să devină invariant la perturbații geometrice. Rezultatul procesului este vizibil în figura 6.3.



(a) Caracter înainte de aplicarea rotației



(b) Caracter după aplicarea rotației

Figura 6.3: Paralelă între un caracter înainte și după transformare

Variantele augmentate sunt salvate alături de originale, marcate printr-un sufix dedicat (`_aug_rot`), ceea ce permite distingerea ulterioară a mostrelor sintetice de cele reale. Prin multiplicarea cu factorul de cinci, setul de antrenare crește de la 1862 la 9310 mostre.

Setul de date rezultat

Prin consolidarea setului de caractere etichetate manual, cu sursa externă de date, a rezultat configurația finală a corpusului de antrenare. Frecvența de apariție a fiecărei clase în cadrul setului final este ilustrată în Figura 6.4.

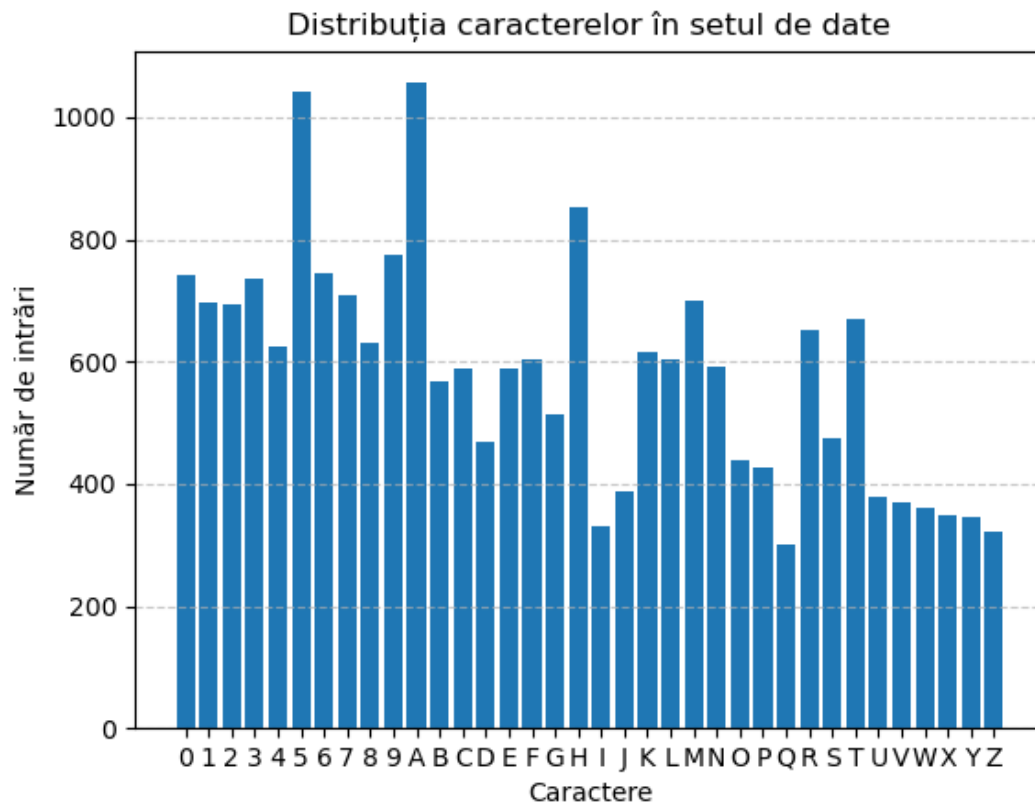


Figura 6.4: Distribuția caracterelor în setul de date final

6.2 Testare și metrice

Evaluarea sistemului urmărește două niveluri complementare: performanța componente de recunoaștere a caracterelor, considerată izolat, și performanța întregului lanț de procesare, măsurată de la imaginea de intrare până la șirul de caractere recunoscut.

6.2.1 Performanța componente de recunoaștere a caracterelor

Pentru evaluarea clasificatorului de caractere, setul de date este împărțit într-un subset de antrenare și unul de testare, în proporție de 80%, respectiv 20%. Împărțirea este stratificată, astfel încât distribuția claselor să fie păstrată în ambele subseturi.

Metricile raportate pentru clasificator sunt acuratețea globală și raportul de clasificare, care cuprinde, pentru fiecare clasă, precizia (*precision*), rapelul (*recall*) și scorul F_1 . Aceste metrice sunt completate de matricea de confuzie, utilă pentru identificarea perechilor de caractere frecvent confundate (de exemplu cifra 0 și litera O, sau cifra 1 și litera I).

În urma antrenării clasificatorului statistic asupra setului de date menționat anterior au fost obținute următoarele rezultate generale pe subsetul de testare format din 4192 intrări:

- **Acuratețe globală:** 96%,
- **Precizie medie:** 96%,
- **Rapel (*Macro Recall*):** 95%,
- **Scor mediu F1:** 96%

Apropierea dintre valorile medii macro și acuratețea globală indică o performanță uniformă pe ansamblul claselor, fără simboluri defavorizate. Acest comportament este așteptat, întrucât setul combinat menține un volum suficient de exemplare pentru majoritatea claselor, evitând dezechilibrele. În ansamblu, o acuratețe de 96% confirmă că reprezentarea liniarizată a imaginii, combinată cu nucleul RBF (*Radial Basis Function* - Funcție de bază radială), captează suficientă informație distinctivă pentru separarea celor 36 de clase.

Structura matricei de confuzie poate fi observată în figura 6.5. Erorile sunt rare și concentrate într-un număr restrâns de perechi de simboluri. Deși rare, erorile persistă în urma procesului de antrenare, în mod special la perechile de caractere care prezintă o similitudine a formei geometrice: pentru perechea formată din cifra 0 și litera O există 14 erori care apar în mod simetric. De asemenea, există confuzii pentru următoarele perechi: litera G și C, cifra 1 și litera I sau T.

Aceste confuzii nu au, însă, o pondere uniformă în contextul aplicației. Confuziile de tip cifră-literă, precum 0-O sau 1-I, sunt în principiu recuperabile: poziția unui simbol în cadrul numărului de înmatriculare determină, conform formatului, dacă acesta trebuie să fie o cifră sau o literă, astfel încât o etapă ulterioară de decodare de format poate valorifica această cunoștință apriori pentru a le clarifica. În schimb, confuziile de tip literă-literă (G-C, I-T) rămân nerezolvabile și constituie limita erorilor clasificatorului.

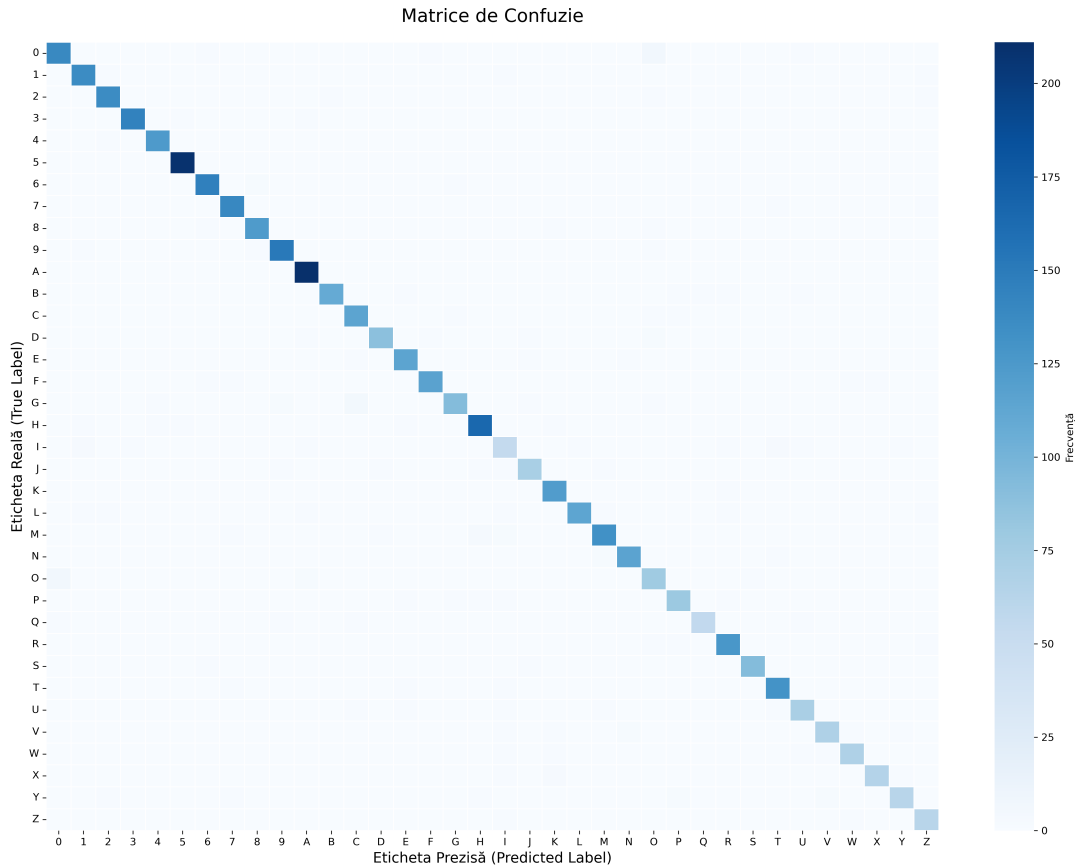


Figura 6.5: Matricea de confuzie obținută în urma antrenării clasificatorului

6.2.2 Performanța întregului sistem

Pe lângă evaluarea izolată a clasificatorului, sistemul este evaluat și la nivel de etapă: rata de localizare corectă a plăcuței, rata de segmentare corectă a caracterelor și, în final, metrica integrată (*end-to-end*), reprezentată de proporția plăcuțelor citite corect în întregime. Această din urmă metrică reflectă cel mai fidel performanța percepută a sistemului, întrucât o singură eroare la nivel de caracter invalidează întregul rezultat.

Înainte de a continua cu detaliile tehnice și numerice ale metricilor, este necesară clarificarea metodologiei de evaluare. După cum am menționat în cadrul secțiunii dedicate seturilor de date, colecția destinată evaluării sistemului cuprinde două componente: una internațională, formată din numere de înmatriculare din Thailanda, și una locală, cu numere din România. Astfel, pentru a obține rezultate fidele asupra performanței, utilitarul software `src/tools/evaluate.py` a fost rulat în mod separat asupra celor două componente.

Rularea separată a fost necesară și pentru că modulul de recunoaștere a caracterelor a fost adaptat să valorifice o cunoștință apriori esențială: structura formatului numărului de înmatriculare. Întrucât poziția fiecărui simbol determină, conform

formatului regional, dacă acesta trebuie să fie cifră sau literă, metoda `predict` a clasei `CharacterRecognition` înlocuiește predicția top-1 a SVM-ului cu marginile furnizate de `decision_function` și restrânge subsetul de clase admis de format. Această decodare condiționată recuperează tocmai confuziile de tip cifră-literă discutate anterior (0-O, 1-I), care altfel ar invalida întregul șir.

Evaluarea localizării corectă a plăcuței

Pentru a putea cuantifica proporția în care sistemul extrage în mod corect regiunea de interes ce conține numărul de înmatriculare, a fost folosită metrica *Intersecție supra Reuniune* (IoU - *Intersection over Union*). Pentru a calcula această metrică, este nevoie atât de regiunile delimitate de sistem drept zone candidate, cât și de coordonatele reale ale plăcuței (regiunea adnotată manual).

Deoarece modulul de localizare propune mai multe regiuni candidat pentru o singură imagine, evaluarea performanței se face doar raportat la zona cu cel mai mare raport IOU. Astfel, o imagine pentru care există cinci predicții posibile pentru regiunea de interes, va fi redusă la un singur rezultat IOU. Această abordare a fost aleasă deoarece, precum am menționat în capitolul anterior, modulul de segmentare extrage implicit zona ideală, mai exact zona cu cel mai mare IOU.

În urma rulării celor două utilitare software, `src/tools/evaluate.py` și `src/tools/metrics.py`, modulul de localizare obține un IoU mediu de ≈ 0.65 pe componenta internațională și ≈ 0.63 pe cea locală. Valorile moderate se explică prin natura modulului de extracție: etapele de dilatare orizontală și de fuziune a candidaților extind în mod deliberat regiunea propusă, pentru a garanta că plăcuța nu este fragmentată și că toate caracterele rămân în interiorul zonei de interes. Deoarece metrica IoU penalizează în egală măsură surplusul și deficitul de suprafață, o margine de câțiva pixeli în jurul plăcuței reduce sensibil scorul, chiar și atunci când localizarea este corectă din punct de vedere funcțional. Această explicație este confirmată practic de creșterea scorului IoU la ≈ 0.80 în momentul introducerii unei etape de validare cu un prag minim de încredere de 0.50.

Evaluarea segmentării caracterelor

Spre deosebire de localizare, pentru care există o adnotare manuală a regiunii de interes, etapa de segmentare nu dispune de o delimitare de referință a fiecărui caracter în parte. Din acest motiv, evaluarea segmentării se realizează indirect, prin compararea numărului de caractere extrase de sistem cu lungimea șirului de referință al plăcuței. Metrica adoptată este acordul la nivel de număr de caractere: pentru fiecare imagine, raportul dintre minimul și maximum celor două valori, după cum este prezentat în Ecuația 6.1.

$$\text{Acord} = \frac{\min(n_{\text{seg}}, n_{\text{gt}})}{\max(n_{\text{seg}}, n_{\text{gt}})} \quad (6.1)$$

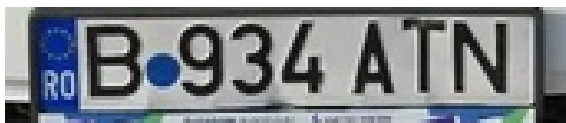
unde n_{seg} este numărul de caractere segmentate, iar n_{gt} este numărul de simboluri din adevărul de referință. Valoarea 1 indică un acord perfect, iar abaterile, fie prin sub-segmentare, fie prin supra-segmentare reduc proporțional scorul. Această metrică surprinde corectitudinea structurală a segmentării fără a presupune o adnotare la nivel de caracter.

Pentru componenta internațională, etapa de segmentare a obținut un acord mediu de ≈ 0.85 , iar pentru cea locală de ≈ 0.82 . Interpretarea acestor valori sugerează că, în mare parte, modulul de segmentare extrage un număr apropiat de caractere relativ la sursa de adevăr. Diferența rămasă până la acordul perfect este atribuită, pe de o parte, plăcuțelor pentru care etapa de localizare a furnizat un decupaj imperfect, iar pe de altă parte cazurilor de fuziune a caracterelor adiacente sau de fragmentare a caracterelor cu structură discontinuă.

Evaluare E2E

Metrica integrată (*end-to-end*) reflectă cel mai fidel performanța percepută a sistemului, întrucât consideră corectă o plăcuță doar atunci când întregul șir de caractere coincide cu adevărul de referință. O singură eroare la nivel de caracter, indiferent de poziția sa, invalidează rezultatul. Pentru a completa această măsură binară, evaluarea raportează și *similaritatea Levenshtein* a șirului recunoscut față de cel de referință, o metrică mai fină care surprinde gradul de corectitudine parțială a predicțiilor invalidate.

Evaluarea pe componenta internațională a setului de date a indicat o acuratețe de $\approx 0,69$, în timp ce pentru componenta națională s-a obținut un procentaj de $\approx 0,73$. Similaritatea Levenshtein medie depășește în ambele cazuri pragul de $\approx 0,80$. Această valoare, semnificativ mai ridicată decât rata de potrivire exactă, confirmă faptul că o proporție importantă a predicțiilor invalidate diferă de adevărul de referință printr-un singur caracter. Cel mai adesea, aceste erori punctuale sunt cauzate de posibilele confuzii vizuale ale modulului de recunoaștere, sau diferitele artefacte vizuale prezente pe o plăcuță, evidențiate în Figura 6.6



(a) Bulină albastră care va fi segmentată în mod incorect drept un caracter



(b) Artefact care blochează parțial numărul de înmatriculare

Figura 6.6: Exemple de artefacte ce pot deteriora procesul de recunoaștere

6.3 Experimente

Experimentele prezentate în această secțiune urmăresc să justifice, prin rezultate cantitative, deciziile de proiectare adoptate în fluxul de procesare și să evidențieze contribuția fiecărei componente la performanța de ansamblu. Ele sunt organizate în trei părți: comparația dintre două abordări ale etapei de localizare, comparația dintre cele două abordări ale etapei de segmentare și seria de îmbunătățiri progresive aduse fluxului de recunoaștere.

6.3.1 Comparația abordărilor de localizare

Înainte de adoptarea abordării bazate pe operatorul Sobel și pe operațiile morfologice descrise anterior, etapa de localizare a fost implementată într-o manieră similară, dar cu un discriminant principal al regiunilor complet diferit: varianta inițială se baza pe aproximarea poligonală a contururilor.

Lanțul de preprocesare urma aceiași pași ca varianta curentă: conversia în tonuri de gri a cadrului primit de la modulul de intrare, atenuarea zgomotului și binarizarea, cu două deosebiri: reducerea zgomotului se realiza printr-un operator diferit de filtrul median, iar accentuarea muchiilor folosea detectorul de contururi Canny în locul operatorului Sobel. Extragerea contururilor (`findContours`) era identică cu cea din varianta curentă, însă selecția primilor `top_k` candidați se făcea pe baza aproximării poligonale (`approxPolyDP`). Acest discriminant s-a dovedit însă nefiabil: în practică, multe dintre plăcuțe erau extrase ca poligoane cu un număr ridicat de laturi, ratând astfel criteriul geometric al formei dreptunghiulare. Tabelul 6.1 sintetizează diferența de performanță dintre cele două abordări.

Metodă de localizare	IoU mediu
Aproximare poligonală (<code>approxPolyDP</code> , variantă explorată)	< 0,40
Sobel + morfologie + <code>top_k</code> (abordarea propusă)	≈ 0,60

Tabela 6.1: Comparație între cele două abordări ale etapei de localizare, evaluate prin același lanț de selecție `top_k` și corecție de înclinare.

Abordarea bazată pe aproximare poligonală nu a reușit să depășească un IoU mediu de 0,40, în timp ce abordarea curentă atinge aproximativ 0,60. Cauza principală a limitării este dependența de detectarea unui contur închis și dreptunghiular al marginii plăcuței: în condiții reale, această margine este frecvent fragmentată, astfel încât aproximarea poligonală fie nu produce un patrulater, fie îl produce deplasat față de plăcuța reală. Spre deosebire de aceasta, abordarea bazată pe Sobel și operații morfologice nu depinde de conturul marginii plăcuței, ci de concentrarea

de muchii verticale generate de caractere: un semnal mai robust și independent de calitatea conturului exterior.

6.3.2 Comparația abordărilor de segmentare

Pentru etapa de segmentare au fost considerate două dintre abordările prezentate în capitolul 3: segmentarea bazată pe proiecții orizontale și verticale și cea bazată pe contururi, ghidată de cunoștințe apriori privind aspectul caracterelor. Înainte de detalierea tehnică a celor două, tabelul 6.2 sintetizează diferența de performanță dintre ele, diferență care a fundamentat alegerea abordării pe contururi.

Metodă de segmentare	Localizare	Segmentare	End-to-end
Bazată pe proiecții (variantă explorată)	0,65	0,20	0,05
Bazată pe contururi (implementarea propusă)	0,65	0,85	0,69

Tabela 6.2: Comparație directă între cele două abordări ale etapei de segmentare, pe același set de candidați produși de etapa de localizare.

Aceste diferențe uriașe nu provin dintr-un avantaj al setului care să favorizeze metodele bazate pe contururi, ci din modul în care fiecare abordare tratează semnalul.

Analiza proiecțiilor comprimă imaginea binară într-un profil unidimensional, obținut prin însumarea pixelilor de prim-plan pe coloane, iar separarea caracterelor depinde de existența unor văi suficient de adânci între vârfurile asociate fiecărui simbol. Pe plăcuțele din setul de evaluare, această condiție este rar îndeplinită: reziduurile cadrului (banda UE, șuruburile de prindere) și, mai ales, înclinarea de câteva grade a plăcuței determină suprapunerea pe verticală a coloanelor caracterelor adiacente, astfel încât văile dintre ele nu mai coboară sub pragul de extragere a vârfurilor.

Abordarea bazată pe contururi nu este afectată de acest fenomen, întrucât decide la nivelul fiecărei componente conexe în parte, izolând fiecare caracter prin propriul contur închis, independent de unghiul de înclinare și de spațierea dintre simboluri.

6.3.3 Îmbunătățiri progresive ale fluxului de recunoaștere

Plecând de la configurația de bază descrisă în secțiunile anterioare, au fost adoptate, succesiv, cinci intervenții incrementale care vizează etapele de segmentare, decodare și clasificare. Evaluarea s-a făcut pe setul de date dedicat evaluării sistemului, iar metricile raportate sunt rata *end-to-end* (egalitate exactă), similaritatea Le-

venshtein a OCR-ului, acordul la nivel de număr de caractere segmentate și IoU-ul localizării. Tabelul 6.3 sintetizează contribuția cumulativă a fiecărei intervenții.

Configurație	End-to-end	OCR sim.	Acord seg.	IoU
Bază	0,489	0,768	0,809	0,588
+ reconstrucția segmentării	0,597	0,801	0,839	0,588
+ decodare condiționată de format	0,645	0,801	0,839	0,588
+ trăsături HOG pentru SVM	0,667	0,807	0,839	0,588
+ pre-procesare cu conservarea proporțiilor	0,677	0,806	0,839	0,588
+ padding LP și augmentare cu rotații	0,694	0,813	0,843	0,589

Tabela 6.3: Contribuția cumulativă a celor cinci intervenții. Fiecare linie include modificările liniilor anterioare. Cumulat, rata end-to-end a crescut cu 20,5 puncte procentuale ($91 \rightarrow 129$ plăcuțe corecte din 186).

Reconstrucția etapei de segmentare

Înainte de intervenție, decupajul de plăcuță era binarizat direct, iar contururile externe erau filtrate exclusiv prin raportul de aspect w/h , cu un prag inferior de 0,15. Matricea de confuzii a arătat două surse sistematice de erori: aproximativ 250 de caractere lipsă (dominate de $5 \rightarrow \emptyset$ cu 55 de cazuri, $1 \rightarrow \emptyset$ cu 48 de cazuri) și aproximativ 30 de caractere inserate fals ($\emptyset \rightarrow A$, $\emptyset \rightarrow T$, $\emptyset \rightarrow H$). Caracterul 1, cu un raport real w/h între 0,07 și 0,12, era respins în întregime de pragul geometric, iar reziduurile cadrului plăcuței (banda UE, șuruburile de prindere) produceau contururi care imitau caractere reale.

Soluția a constatat în (1) restrângerea decupajului la cea mai mare componentă conexă deschisă din interiorul plăcuței, eliminând cadrul înainte de binarizare; (2) coborârea pragului w/h la 0,08, recuperând caracterul 1; (3) respingerea contururilor care ating marginile laterale ale plăcuței tăiate, anulând patternul de inserții; (4) o trecere de consistență pe înălțimi, care elimină casetele aflate în afara intervalului $[0,55 m, 1,45 m]$, unde m este înălțimea mediană a supraviețuitorilor; (5) un divizor de blocuri largi care, pentru casete cu $w > 1,2 h$, taie la coloana cu cea mai joasă proiecție verticală, recuperând caracterele lipite. Inserțiile au scăzut de la ~ 30 la ~ 5 , iar rata end-to-end a urcat cu 10,8 puncte procentuale.

Au fost respinse explicit aplicarea unei închideri morfologice 3×3 , care, pe plăcuțe de ~ 27 de pixeli înălțime, a unit caracterele adiacente (numărul de contururi crude a scăzut de la 23 la 4 într-un caz observat), și varianta verticală (2, 1), cu același efect în formă atenuată.

Decodare condiționată de format

Analiza matricii de confuzie a indicat substituții frecvente între clase vizual similare ($4 \rightarrow F$, $0 \rightarrow O$, $1 \rightarrow I$), generate de ambiguitatea internă a SVM-ului. Indiferent de regiune, structura numărului de înmatriculare oferă tocmai acest context: formatul fixează, pentru fiecare poziție, dacă simbolul așteptat este o cifră sau o literă. Cunoștințele pre-existente asupra formatului pot facilita procesul de clasificare, restrângând domeniul de predicții posibile.

Modificarea, localizată în metoda `predict` a clasei `CharacterRecognition`, este formulată independent de un format anume. Ea înlocuiește predicția top-1 a SVM-ului cu marginile furnizate de `decision_function` și restrânge și restrânge subșetul de clase admis de format.

Atunci când o lungime dată admite mai multe formate candidate, fiecare șablon este evaluat, iar cel cu marginea medie pe poziție cea mai mare este selectat. Pentru lungimile care nu corespund niciunui format, decodarea revine la varianta neconstrânsă, evitând orice regresie. Aceeași logică se aplică nemodificat atât plăcuțelor românești și thailandeze.

Trăsături HOG pentru clasificator

Reprezentarea pe 784 de dimensiuni (intensitățile brute ale pixelilor 28×28) a fost înlocuită cu un descriptor HOG.

Modelul a fost re-antrenat, atingând o acuratețe de 97% pe subșetul independent. În pipeline-ul integrat, rata *end-to-end* a urcat cu 2,2 puncte procentuale (la 66,7%). Un alt rezultat este scăderea numărului de perechi de erori din matricea de confuzie de la 82 la 70, indicând o concentrare a acestora în categorii mai bine definite și mai ușor de abordat în continuare.

Păstrarea proporțiilor caracterului

Ultima modificare rezolvă o limitare a etapei de pregătire a caracterului: redimensionarea directă la 28×28 deforma raportul de aspect al claselor înguste. Un caracter 1, cu $w/h \approx 0,12$ la segmentare, devenea un pătrat, iar descriptorul HOG raporta o distribuție de orientări similară cu cea a unui I, T sau L. Modificarea din logica funcției `preprocess_char` completează imaginea cu o margine constantă până la un pătrat, înainte de redimensionarea la 28×28 .

Augmentare cu rotații a setului de antrenare

Schimbarea descrisă în cele ce urmează are drept scop creșterea robusteții clasificatorului la diferența dintre distribuția setului de antrenare și cea din mediul

de execuție. Această ajustare a urcat rata *end-to-end* cu 1,7 puncte procentuale (la 69,4%).

Intervenția constă într-un script de augmentare (`src/augment_dataset.py`) care, pentru fiecare exemplar etichetat din setul obținut în mod manual, generează patru variante rotite la $\pm 3^\circ$ și $\pm 5^\circ$ în jurul centrului, cu marginea completată prin aceeași metodă `preprocess_char`). Setul de antrenament crește astfel de la 1862 la 9310 mostre.

Bilanț

Cumulat, cele cinci intervenții au urcat rata *end-to-end* de la 48,9% la 69,4%. Două limitări reziduale rămân vizibile: tăierea caracterului inițial al plăcuței, cauzată în etapa de localizare și un set restrâns de substituții vizual ambigue dintre litere sau dintre cifre ($Y \rightarrow M$, $F \rightarrow 6$, $1 \rightarrow I$).

Capitolul 7

Concluzii și perspective de viitor

Acest capitol final recapitulează contribuțiile lucrării și rezultatele obținute, după care discută limitările sistemului și direcțiile în care acesta poate fi extins.

7.1 Concluzii

Obiectivul lucrării a fost construirea unui sistem de recunoaștere automată a numerelor de înmatriculare care să funcționeze fiabil pe o varietate largă de formate și să respecte o constrângere de timp real, folosind exclusiv tehnici clasice de viziune artificială și un clasificator SVM, fără a recurge la rețele neuronale profunde. Sistemul IORA, structurat pe trei etape: localizare, segmentare și recunoaștere, arată că o astfel de abordare rămâne viabilă atunci când este completată cu cunoștințe apriori despre structura problemei.

Un prim aspect confirmat de experimente este superioritatea criteriilor geometrice față de cele dependente de aparență. Renunțarea la segmentarea pe culoare a permis generalizarea la cele patru combinații cromatice admise în România și la formatele internaționale, întrucât raportul de aspect al regiunii este un descriptor mult mai stabil decât o paletă cromatică fixă. Aceeași concluzie se confirmă la nivelul segmentării: decizia luată independent pe fiecare componentă conexă rămâne robustă la înclinarea plăcuței, în timp ce analiza proiecțiilor cedează în prezența vailor estompate dintre caractere (un acord de 0,85 față de 0,20).

Al doilea aspect ține de rolul decisiv al cunoștințelor apriori. În absența unui model profund, performanța depinde de modul în care este exploatată structura numărului de înmatriculare. Decodarea condiționată de format, care restrânge la fiecare poziție mulțimea etichetelor admise, recuperează confuziile de tip cifră-literă (0-O, 1-I) care altfel ar invalida întregul șir. Tot pe cunoștințe apriori se sprijină și cuplajul dintre localizare și segmentare: returnarea mai multor candidați de plăcuță transformă etapa de segmentare într-un mecanism de recuperare a erorilor de loca-

lizare, fără un cost de calcul suplimentar semnificativ.

Cantitativ, clasificatorul SVM antrenat pe descriptori HOG cu nucleu RBF a atins o acuratețe de 96% pe cele 36 de clase de caractere, iar seria de intervenții incrementale aplicate fluxului a urcat rata *end-to-end* de la 48,9% la 69,4%. Similaritatea Levenshtein medie depășește 0,80, ceea ce arată că majoritatea erorilor rămase se reduc la un singur caracter greșit. Dincolo de metrici, sistemul este integrat într-o aplicație funcțională, organizată pe trei module decuplate prin interfețe și structuri de date proprii, care poate fi rulată atât pe un sistem integrat, cât și ca serviciu web, fără modificarea nucleului de procesare.

Merită subliniat că aceste rezultate au fost obținute pe un set de evaluare eterogen. Deși componentele locală și internațională au fost curățate și normalizate manual, ele rămân departe de a fi uniforme: persistă diferențe importante de unghi de captură, distanță, iluminare, font și calitate a imaginii. Într-un scenariu de exploatare precum cel vizat, un punct de control al accesului auto într-o parcare, aceste variabile sunt, în schimb, fixe și cunoscute: poziția camerei, distanța focală, unghiul și dimensiunea relativă a plăcuței nu variază de la un cadru la altul. Într-un asemenea cadru controlat, în care sistemul poate fi calibrat o singură dată pentru condițiile concrete de instalare, performanța reală ar fi sensibil mai ridicată decât cea măsurată pe setul eterogen de evaluare, care reprezintă mai degrabă un scenariu pesimist.

7.2 Perspective de viitor

O primă direcție o constituie alcătuirea unui set de date public și reprezentativ pentru plăcuțele din România, în prezent inexistent; componenta locală folosită aici a fost colectată manual, ceea ce îi limitează volumul și diversitatea. O a doua direcție vizează limitările reziduale observate în experimente: tăierea ocazională a primului caracter al plăcuței, cu origine în etapa de localizare, și un set restrâns de substituții vizual ambigue dintre litere sau dintre cifre, care nu pot fi corectate prin decodarea de format. Introducerea unei etape de validare cu un prag minim de încredere, care a ridicat scorul IoU al localizării la $\approx 0,80$, reprezintă o cale concretă de îmbunătățire în acest sens. În fine, componenta clasică, eficientă din punct de vedere computațional, ar putea fi completată de un clasificator neuronal ușor, dedicat exclusiv caracterelor dificile, păstrând constrângerea de execuție în timp real pe *hardware* modest.

Per ansamblu, lucrarea arată că tehnicile clasice de viziune artificială, atunci când sunt ghidate de cunoștințe apriori solide, pot susține un sistem de recunoaștere a numerelor de înmatriculare suficient de robust și de rapid pentru un scenariu real de control al accesului, oferind totodată o bază clară pentru dezvoltările ulterioare.

Bibliografie

- [1] Jithmi Shashirangana, Heshan Padmasiri, Dulani Meedeniya, and Charith Perera. Automated license plate recognition: a survey on methods and techniques. *IEEE Access*, 9:11203–11225, 2020.
- [2] Xifan Shi, Weizhong Zhao, and Yonghang Shen. Automatic license plate recognition system based on color image processing. In *International conference on computational science and its applications*, pages 1159–1168. Springer, 2005.
- [3] Shyang-Lih Chang, Li-Shien Chen, Yun-Chung Chung, and Sei-Wan Chen. Automatic license plate recognition. *IEEE transactions on intelligent transportation systems*, 5(1):42–53, 2004.
- [4] Lin Luo, Hao Sun, Weiping Zhou, and Limin Luo. An efficient method of license plate location. In *2009 First International Conference on Information Science and Engineering*, pages 770–773. IEEE, 2009.
- [5] V Karthikeyan and VJ Vijayalakshmi. Localization of license plate using morphological operations. *arXiv preprint arXiv:1402.5623*, 2014.
- [6] Abbas M Al-Ghaili, Syamsiah Mashohor, Alyani Ismail, and Abdul Rahman Ramli. A new vertical edge detection algorithm and its application. In *2008 international conference on computer engineering & systems*, pages 204–209. IEEE, 2008.
- [7] Tran Duc Duan, TL Hong Du, Tran Vinh Phuoc, and Nguyen Viet Hoang. Building an automatic vehicle license plate recognition system. In *Proc. Int. Conf. Comput. Sci. RIVF*, volume 1, pages 59–63, 2005.
- [8] R Bremananth, A Chitra, Vivek Seetharaman, and Vidya Sudhan L Nathan. A robust video based license plate recognition system. In *Proceedings of 2005 International Conference on Intelligent Sensing and Information Processing*, 2005., pages 175–180. IEEE, 2005.
- [9] Christos Nikolaos E Anagnostopoulos, Ioannis E Anagnostopoulos, Vassilis Loumos, and Eleftherios Kayafas. A license plate-recognition algorithm for

- intelligent transportation system applications. *IEEE Transactions on Intelligent transportation systems*, 7(3):377–392, 2006.
- [10] Kenji Kanayama, Yoshimasa Fujikawa, Koichi Fujimoto, and Masanobu Horino. Development of vehicle-license number recognition system using real-time image processing and its application to travel-time measurement. In *[1991 Proceedings] 41st IEEE Vehicular Technology Conference*, pages 798–804. IEEE, 1991.
- [11] Zhang Sanyuan, Zhang Mingli, and Ye Xiuzi. Car plate character extraction under complicated environment. In *2004 IEEE International Conference on Systems, Man and Cybernetics (IEEE Cat. No. 04CH37583)*, volume 5, pages 4722–4726. IEEE, 2004.
- [12] Abdulkerim Capar and Muhittin Gokmen. Concurrent segmentation and recognition with shape-driven fast marching methods. In *18th International Conference on Pattern Recognition (ICPR'06)*, volume 1, pages 155–158. IEEE, 2006.
- [13] James A Sethian. A fast marching level set method for monotonically advancing fronts. *Proceedings of the National Academy of Sciences*, 93(4):1591–1595, 1996.
- [14] Muhammad Sarfraz, Mohammed Jameel Ahmed, and Syed A Ghazi. Saudi arabian license plate recognition system. In *2003 International conference on geometric modeling and graphics, 2003. Proceedings*, pages 36–41. IEEE, 2003.
- [15] Eun Ryung Lee, Pyeoung Kee Kim, and Hang Joon Kim. Automatic recognition of a car license plate using color image processing. In *Proceedings of 1st international conference on image processing*, volume 2, pages 301–305. IEEE, 1994.
- [16] Frank Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological review*, 65(6):386, 1958.
- [17] Vladimir N Vapnik. Pattern recognition using generalized portrait method. *Automation and remote control*, 24(6):774–780, 1963.
- [18] Vladimir N Vapnik. A note on one class of perceptrons. *Automat. Rem. Control*, 25:821–837, 1964.
- [19] Vladimir N Vapnik and A Ya Chervonenkis. On the uniform convergence of relative frequencies of events to their probabilities. In *Measures of complexity: festschrift for alexey chervonenkis*, pages 11–30. Springer, 2015.
- [20] Nello Cristianini and John Shawe-Taylor. *An Introduction to Support Vector Machines and Other Kernel-based Learning Methods*. Cambridge University Press, 2000.

- [21] Leslie G Valiant. A theory of the learnable. *Communications of the ACM*, 27(11):1134–1142, 1984.
- [22] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine learning*, 20(3):273–297, 1995.
- [23] Bernhard E Boser, Isabelle M Guyon, and Vladimir N Vapnik. A training algorithm for optimal margin classifiers. In *Proceedings of the fifth annual workshop on Computational learning theory*, pages 144–152, 1992.
- [24] Anil K Jain, Jianchang Mao, and K Moidin Mohiuddin. Artificial neural networks: A tutorial. *Computer*, 29(3):31–44, 1996.
- [25] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feed-forward networks are universal approximators. *Neural networks*, 2(5):359–366, 1989.
- [26] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- [27] *CODE COMPLETE 2nd Edition*. WP Publishers & Distributors Pvt Limited, 2004.
- [28] Roboflow Universe Projects. License plate recognition dataset. <https://universe.roboflow.com/roboflow-universe-projects/license-plate-recognition-rxg4e>, 2022. accesat iunie 2026.
- [29] T. E. de Campos, B. R. Babu, and M. Varma. Character recognition in natural images. In *Proceedings of the International Conference on Computer Vision Theory and Applications, Lisbon, Portugal, February 2009*.